Ref. Ares(2025)8326393 - 02/10/2025

Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project funded by the European Innovation Council and SMEs Executive Agency

D3.1 Local risk measures for multi-agent systems

Work Package: WP3

Authors: Ruohan Wang, Sofie Haesaert, Zengjie Zhang, Eleftherios Vlahakis, Dimos V. Dimarogonas

Project Title: Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project Acronym: SymAware

Grant Agreement number: 101070802

Project Start Date: 1st October 2022

Duration: 42 months

Project Coordinator: Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Project Members:

Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Technische Universiteit Eindhoven (TUE)

Kungliga Tekniska Hoegskolan (KTH)

Uppsala Universitet (UU)

Stichting Koninklijk Nederlands Lucht - En Ruimtevaartcentrum (NLR)

Siemens Industry Software Netherlands BV (SISW)

Document Identifier: D3.1

Issue: 1.0

Issue Date: 30.09.2025

Prepared by: Technische Universiteit Eindhoven (TUE)

Approved: Yes

Dissemination Status: Public

History Chart:

Issue	Date (DD/MM/YYYY)	Updated Sections	Cause of Change	Implemented by
1.0	29.09.2025	All sections	New Document	Ruohan Wang Sofie Haesaert
1.0	30.09.2025	All sections	Review by Arabinda Ghosh	Ruohan Wang Sofie Haesaert Eleftherios Vlahakis

The information in this document is subject to change without notice.

All rights reserved

The document is proprietary of the SymAware consortium members listed on the front page of this document. The document is supplied on the express understanding that it is to be treated as confidential and may not be used or disclosed to others in whole or in part for any purpose except as expressly authorized in terms of Grant Agreement number 101070802. The SymAware consortium makes no warranty for the information contained in this document; neither does it assume any legal liability or responsibility for the accuracy, completeness or usefulness of this information. Company or product names mentioned in this document may be trademarks or registered trademarks of their respective companies.

Contents

List of Figures **4**

List of Tables **5**

1 Introduction **6**
1.1 SymAware: High-level Description of the Project . . . 6
1.2 Task 3.1 . . . 7

2 Background and Problem Formulation **8**
2.1 Problem Setup . . . 8
2.1.1 Specifications . . . 9
2.2 Problem Formulation . . . 10
2.3 Approach . . . 11

3 Risk-Aware Real-Time Task Allocation for Stochastic Multi-Agent Systems: A Stochastic MPC Approach **12**
3.1 Introduction . . . 12
3.2 Preliminaries and Problem Statement . . . 13
3.2.1 Multi-Agent Systems . . . 14
3.2.2 Signal Temporal Logic & Probabilistic Satisfaction . . . 14
3.2.3 Problem Statement & Approach . . . 15
3.3 Risk-Aware Multi-Agent Task Allocation and Control Synthesis . . . 16
3.3.1 Decomposing Agent Dynamics . . . 16
3.3.2 Specification Decomposition . . . 17
3.3.3 Heuristic Agent-Specification Filtering . . . 18
3.3.4 Control Synthesis and Local Risk Value . . . 19
3.3.5 Task Allocation Algorithm . . . 20
3.3.6 Implementation & Theoretical Analysis . . . 21
3.4 Case Study . . . 22
3.5 Connection to Complementary Work and Conclusions . . . 23
3.5.1 Splitting Specifications in Time . . . 23
3.5.2 Direct-data Driven Approach to MPC . . . 24
3.5.3 Conclusions . . . 24

4 Efficient Risk Quantification of Multi-Agent Systems via Tensorized DP and Simulation Relations **25**
4.1 Introduction . . . 25
4.1.1 Problem Statement . . . 26
4.2 Tensor-based efficient risk quantification for finite MDPs . . . 27
4.2.1 Value Iteration for Temporal Logic Specifications . . . 27
4.2.2 Improved Scalability: Tensorized Dynamic Programming . . . 28
4.2.3 Computational Tensor Tree Expanded via $\mathbf{T}^{\pi_q^{(i)}}_{\alpha^{(i)}}$ . . . 30
4.2.4 Local Policy Optimization and Local Satisfaction Probabilities . . . 32
4.2.5 Scalability Benchmarking . . . 33
4.3 Tensor-based Efficient Risk Quantification for Continuous State and Model Uncertainty . . . 35
4.3.1 Simulation-Relation based Approaches for Uncertain Stochastic Systems . . . 35
4.3.2 Tensor-Based Approach . . . 37
4.4 Complementary Work on Scalability: Heterogeneous Tooling . . . 39
4.5 Conclusions . . . 39

5 Conclusions **41**

List of Figures

3.1 Decomposing and allocating tasks for vehicle routing. 13
3.2 Illustration of the approach at time k: (0) Determine for each agent-specification pair a control strategy and the local risk value. (1) Auction the offers from agent-specification pairs. (2) Make a decision based on the auctioning outcome. 16
3.3 Polytopes H, $B_{\text{in}}(H)$, $B_{\text{out}}(H)$, for Example 1 18
3.4 The shuttle trajectories. 22

4.1 Illustration of order-3 tensor with its Canonical Polyadic Decomposition 29
4.2 Left: DFA (q_s denotes q_{sink} for simplicity), right: state space labeled by l or α. . . . 30
4.3 Tree expanded iteratively based on Alg. 2 as $\mathcal{G}_0$, $\mathcal{G}_1$, $\mathcal{G}_2$, and $\mathcal{G}_3$. Examples: $\mathcal{G}_0$ with $\mathrm{N}_0 = \{1\}$, $\mathcal{G}_1$ with $\mathrm{N}_1 = \{1, 2, 3\}$ and $\mathcal{E}_1 = \{(1, \alpha, 2), (1, \alpha, 3)\}$. 31
4.4 Satisfaction probability error of rank-1 tree-based value iteration. Error is computed with respect to using exact value iteration. 33
4.5 Effect of varying number of grid cells visualized as satisfaction probabilities on output space (left: 1.6×10^5 grid cells, right: 10^{12} grid cells). 34
4.6 Expanding tree measured by the number of vertices at each iteration step. Each color corresponds to a choice of different order of magnitude in grid cell count. . . . 34
4.7 Scalability analysis (on running time in left y-axis as blue and memory usage in right y-axis as orange) for increasing number of agents. x represents time/memory for ψ_1, ■ for ψ_2. 35
4.8 Contracting credible set Θ and parameter estimate $\hat{\theta}$ for data sizes $N = 10^4, 10^5, 10^6$ (light to dark blue) with an associated confidence bound of $(1 - \alpha) = 0.9$. 36
4.9 Robust satisfaction probability of package delivery case with enlarged spaces when using a multi-layered approach with heterogeneous layers. 39

List of Tables

4.1 Performance summary, including running time (in seconds) and memory usage (in MB) of varying system dimensions and grid counts for specification $\psi := (\neg p_2 \wedge \neg p_3) \mathsf{U} p_1$ 35

Chapter 1

Introduction

1.1 SymAware: High-level Description of the Project

SymAware addresses the fundamental need for a new conceptual framework for awareness in multi-agent systems (MASs) that is compatible with the internal models and specifications of robotic agents and that enables safe simultaneous operation of collaborating autonomous agents and humans. The goal of SymAware is to provide a comprehensive framework for situational awareness to support sustainable autonomy via agents that actively perceive risks and collaborate with other robots and humans to improve their awareness and understanding while fulfilling complex and dynamically changing tasks. The SymAware framework will use compositional logic, symbolic computations, formal reasoning, and uncertainty quantification to characterise and support situational awareness of MAS in its various dimensions, sustaining awareness by learning in social contexts, quantifying risks based on limited knowledge, and formulating risk-aware negotiation of task distributions. These objectives will be achieved in SymAware through

1. logical characterization of awareness using symbolic methods,
2. quantifying the symbolic reasoning for awareness with spatial and temporal ingredients for decision making,
3. risk awareness via quantified knowledge,
4. quantifying and communicating knowledge awareness,
5. demonstrating awareness engineering in aviation and automotive use cases, and
6. identifying requirements for ethical and trustworthy awareness in human-agent interaction.

The objectives of SymAware address the "Awareness Inside" Challenge of the European Innovation Council (EIC) by extending and formalizing human-based models of situational awareness and by providing a novel conceptual situational awareness framework for MASs that encompasses logical characterisation and integrative formal reasoning of interdependent awareness dimensions including knowledge, spatiotemporal, risk and social dimensions. This will support transitioning to safe mixed operation of autonomous agents and humans.

1.2 Task 3.1

Starting from the initial logic and multi-agent model developed in WP1, this task seeks *local risk measures* that (i) are computable locally, (ii) compose into certified global guarantees, and (iii) interact gracefully with learning and awareness.

In other words, the objective of this work package is to develop an appropriate framework for risk assessment in multi-agent systems. Additionally, the framework for risk assessment should be able to take uncertain or data-based knowledge into account. We have been exploring three different approaches or metrics for risk quantification. Risk quantification based on formal abstractions and approximate stochastic simulation relations [1, 2], risk quantification based on stochastic invariant sets and Stochastic MPC [3], and risk quantification embedded in the propositions of the tasks [4, 5]. We have chosen to work with the former two, that is the abstraction-based approach and the stochastic MPC approach.

Scope. We consider stochastic, discrete-time systems abstracted as finite MDPs and temporal logic specifications. We emphasize correct-by-design guarantees, avoiding methods where certification is difficult.

Chapter 2

Background and Problem Formulation

2.1 Problem Setup

Models. Denote a Borel measurable space as $(\mathbb{S}, \mathcal{B}(\mathbb{S}))$ where $\mathbb{S}$ is a set and $\mathcal{B}(\mathbb{S})$ is the Borel sigma-algebra on $\mathbb{S}$. A probability measure $\mathbb{P}$ over $(\mathbb{S}, \mathcal{B}(\mathbb{S}))$ defines the probability space $(\mathbb{S}, \mathcal{B}(\mathbb{S}), \mathbb{P})$ and has realization $s \sim \mathbb{P}$. We assume all such spaces $\mathbb{S}$ are Polish [6]. For a given Polish space $\mathbf{Y}$, a function

$$d : \mathbf{Y} \times \mathbf{Y} \to [0, \infty)$$

is called a *metric* (or *distance*) on $\mathbf{Y}$ if for all $x, y, z \in \mathbf{Y}$ the following hold: 1. (Non-negativity) $d(x, y) \geq 0$. 2. (Identity of indiscernibles) $d(x, y) = 0 \iff x = y$. 3. (Symmetry) $d(x, y) = d(y, x)$. 4. (Triangle inequality) $d(x, z) \leq d(x, y) + d(y, z)$.

System models: Stochastic difference equations We consider a multi-agent system $\mathbf{S}$ composed of agents with fully decoupled dynamics characterized with stochastic difference equations:

$$\begin{aligned} \begin{bmatrix} x_1^+ \\ \vdots \\ x_m^+ \end{bmatrix} &= \begin{bmatrix} f_1(x_1, u_1) \\ \vdots \\ f_m(x_m, u_m) \end{bmatrix} + \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix} \\ \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} &= \begin{bmatrix} h_1(x_1) \\ \vdots \\ h_m(x_m) \end{bmatrix} \end{aligned} \tag{2.1}$$

with the state $x := \begin{bmatrix} x_1^\top & \dots & x_m^\top \end{bmatrix}^\top \in \mathbb{R}^{\sum_{i=1}^m d_{x,i}}$, $d_{x,i} \in \mathbb{N}_+$, the control input $u := \begin{bmatrix} u_1^\top & \dots & u_m^\top \end{bmatrix}^\top \in \mathbb{R}^{\sum_{i=1}^m d_{u,i}}$, $d_{u,i} \in \mathbb{N}_+$, and the state disturbance $w := \begin{bmatrix} w_1^\top & \dots & w_m^\top \end{bmatrix}^\top \in \mathbb{R}^{\sum_{i=1}^m d_{x,i}}$ modeled as white noises, i.e., $w_t \sim \mathcal{N}(0, \Sigma)$ where Σ is a block diagonal positive symmetric matrix and, w_t and $w_{t'}$ are independent for all $t \neq t'$. System (2.1) is initialized with state $x[0] := \begin{bmatrix} x_1[0]^\top & \dots & x_m[0]^\top \end{bmatrix}^\top$.

A stochastic difference equation can also be modeled as a general Markov decision process.

Definition 1 (general Markov decision processes (gMDP))**.** *A discrete-time gMDP is denoted as* $\mathbf{M} := (\mathbb{S}, \mathbb{A}, \mathbb{Y}, \mathbb{T}, s_0, h)$ *with*

- *a Polish state space* $\mathbb{S}$ *with states* $s \in \mathbb{S}$*;*

- *a Polish input space* $\mathbb{A}$ *with inputs/actions* $a \in \mathbb{A}$*;*
- *an output space* $\mathbb{Y}$ *decorated with metric* $\mathbf{d}_{\mathbb{Y}}$*;*
- *a stochastic kernel* $\mathbb{T} : \mathbb{S} \times \mathbb{A} \times \mathcal{B}(\mathbb{S}) \to [0, 1]$*, that assigns to each state-action pair* $s \in \mathbb{S}$ *and* $a \in \mathbb{A}$ *a probability measure* $\mathbb{T}(\cdot \mid s, a)$ *over* $(\mathbb{S}, \mathcal{B}(\mathbb{S}))$*;*
- *an initial state* $s_0 \in \mathbb{S}$*;*
- *a measurable output map* $h : \mathbb{S} \to \mathbb{Y}$*.*

The *execution (state trajectory)* of a gMDP is a sequence of states

$$\boldsymbol{s}_{[0,t]} := \{s[t] | t = 0, 1, \ldots\} \tag{2.2}$$

initialized with the initial state $s[0] \in \mathbb{S}$. In an execution, each consecutive state, $s[t+1] \in \mathbb{S}$, is obtained as a realization $s[t+1] \sim \mathbb{T}(\cdot \mid s[t], a[t])$ of the controlled Borel-measurable stochastic kernel. By applying the output map $h(\cdot)$ over states in the execution, we obtain the associated *output trace* $\boldsymbol{y}_{[0,t]} := \{y[t] \mid t = 0, 1, \ldots\}$. We say that a gMDP is finite if $\mathbb{S}$, $\mathbb{Y}$ and $\mathbb{A}$ are finite.

The system $\mathbf{S}$ (2.1) can be represented as a gMDP $\mathbf{M}$ composed of m decoupled gMDPs $\mathbf{M}^{(i)} = (\mathbb{S}^{(i)}, \mathbb{A}^{(i)}, \mathbb{Y}^{(i)}, \mathbb{T}^{(i)}, s_0^{(i)}, h^{(i)}), i \in \{1, 2, \ldots, m\}$, that is

$$\mathbf{M} = (\mathbb{S}, \mathbb{A}, \mathbb{Y}, \mathbb{T}, s_0, h) \tag{2.3}$$

with $\mathbb{S} = \prod_{i=1}^{m} \mathbb{S}^{(i)}$, $\mathbb{Y} = \prod_{i=1}^{m} \mathbb{Y}^{(i)}$, $\mathbb{A} = \prod_{i=1}^{m} \mathbb{A}^{(i)}$, $\mathbb{T} = \prod_{i=1}^{m} \mathbb{T}^{(i)}$,$s_0 = (s_0^{(1)}, \ldots, s_0^{(m)})$, $h = \prod_{i=1}^{m} h^{(i)}$,and

$$\mathbb{T}^{(i)}(x_{t+1}^{(i)} | x_t^{(i)}, u_t^{(i)}) = \mathcal{N}(x_{t+1}^{(i)} | f(x_t^{(i)}, u_t^{(i)}), \Sigma[i]).$$

We denote with $\mathbf{C}_{\boldsymbol{\pi}}$ a general control strategy that maps each finite history $s[0], a[0], s[1], a[1], \ldots, s[t]$ to an action $a[t]$. A more specific type of control strategy is a Markov policy.

Definition 2 (Markov policy $\boldsymbol{\pi}$)**.** *A Markov policy* $\boldsymbol{\pi}$ *is a sequence*

$$\boldsymbol{\pi} := \{\pi[t] | t = 0, 1, \ldots\} \tag{2.4}$$

of maps $\pi[t] : \mathbb{S} \to \mathbb{A}$ *that assign an action* $a \in \mathbb{A}$ *to each state* $s \in \mathbb{S}$*.*

We refer to a Markov policy $\boldsymbol{\pi}$ as defined in Definition 2 as *stationary* if $\pi[t]$ does not depend on the time index t, that is for all t it holds that $\pi[t] := \pi$ in (2.4) with $\pi : \mathbb{S} \to \mathbb{A}$. More precisely, we define a *stationary Markov policy* as $\boldsymbol{\pi} := \{\pi, \pi, \ldots\}$. With a slight abuse of notation, π is also used in the rest of the paper to denote a stationary Markov policy. The set of stationary Markov policies is denoted as Π^s.

2.1.1 Specifications

We consider specifications given in linear temporal logics. There are a multitude of different types of temporal logic languages. This includes Linear-time Temporal logic [7] and derived languages such as GR(1) [8], syntactically co-safe Linear Temporal Logics (scLTL) [9]. Additionally for continuous time systems, Signal temporal logic (STL) has been developed [10]. This logic allows for tracking

and monitoring continuous-time specifications. The logic has also been used for specifying robotics tasks [11]. To avoid a multitude of, albeit different, similar definitions we will only give the full definition of scLTL specifications

scLTL. Consider atomic propositions $p_1, p_2, \dots p_N$ that are true or false. The set of atomic propositions and the corresponding alphabet are denoted by $\mathrm{AP} = \{p_1, \dots, p_N\}$ and 2^{AP}, respectively. Each letter $l \in 2^{\mathrm{AP}}$ contains the set of atomic propositions that are true. A (possibly infinite) string of letters forms a word $\boldsymbol{l} := l_0, l_1, \dots$, with associated suffix $\boldsymbol{l}_t := l_t, l_{t+1}, \dots$. The state trajectory $\boldsymbol{s} = s[0], s[1], \dots$ of a system is mapped to the word $\boldsymbol{l} := L(s[0]), L(s[1]), \dots$ using a given labeling function $L : \mathbb{S} \to 2^{\mathrm{AP}}$ that translates each state to a specific letter $l_t = L(s[t])$. Similarly, execution suffixes $\boldsymbol{s}_t := s[t], s[t+1], \dots$ are translated to word suffixes $\boldsymbol{l}_t := l_t, l_{t+1}, \dots$. By combining atomic propositions with logical operators, the language of syntactically co-safe Linear Temporal Logics (scLTL) [9] can be defined as follows.

Definition 3 (scLTL syntax)**.** *An scLTL formula ϕ over a set of atomic propositions* AP*, with $p \in \mathrm{AP}$ has syntax*

$$\phi ::= p \,|\, \neg p \,|\, \phi_1 \wedge \phi_2 \,|\, \phi_1 \vee \phi_2 \,|\, \bigcirc \phi \,|\, \phi_1 \,\mathsf{U}\, \phi_2. \tag{2.5}$$

The semantics of this syntax can be given for the suffixes $\boldsymbol{l}_t$. An atomic proposition $\boldsymbol{l}_t \models p$ holds if $p \in l_t$, while a negation $\boldsymbol{l}_t \models \neg\phi$ holds if $\boldsymbol{l}_t \not\models \phi$. Furthermore, a conjunction $\boldsymbol{l}_t \models \phi_1 \wedge \phi_2$ holds if both $\boldsymbol{l}_t \models \phi_1$ and $\boldsymbol{l}_t \models \phi_2$ are true, while a disjunction $\boldsymbol{l}_t \models \phi_1 \vee \phi_2$ holds if either $\boldsymbol{l}_t \models \phi_1$ or $\boldsymbol{l}_t \models \phi_2$ is true. Also, a next statement $\boldsymbol{l}_t \models \bigcirc\phi$ holds if $\boldsymbol{l}_{t+1} \models \phi$. Finally, an until statement $\boldsymbol{l}_t \models \phi_1 \,\mathsf{U}\, \phi_2$ holds if there exists an $i \in \mathbb{N}$ such that $\boldsymbol{l}_{t+i} \models \phi_2$ and for all $j \in \mathbb{N}, 0 \leq j < i$, we have $\boldsymbol{l}_{t+j} \models \phi_1$. A system satisfies a specification if the generated word $\boldsymbol{l}_0 = L(\boldsymbol{s}_0)$ satisfies the specification, i.e., $\boldsymbol{l}_0 \models \phi$.

2.2 Problem Formulation

Our goal is to synthesize a control strategy for multi-agent systems with decoupled dynamics as given in Eq. (2.1) and to quantify the risks while synthesizing the policy. The control strategy should be synthesized to ensure satisfaction of a given specification ϕ and have a acceptible risk level. Generally, we can describe the risk as a probability.

Problem statement: *Given a specification ϕ and a probability $p \in [0, 1]$, synthesize a control strategy for a set of fully decoupled gMDPs $\mathbf{M}^{(i)}$ such that the controlled system* (2.3) *satisfies ϕ with a lower bound, i.e.,*

$$\mathbf{P}(\mathbf{M} \times \mathbf{C}_{\boldsymbol{\pi}} \models \phi) \geq p. \tag{2.6}$$

Remark 1. *In the above problem statement, the risk is quantified with a simple probability assessment. For more complex and ethically challenging requirements or scenarios the risk assessment can be extended to include weighted and discounted combinations of probabilities of negative events as detailed in our work in [12], which is also executed in the project. For this deliverable, we will focus on the basic risks of a single (positive) event. However, the results also fit the risk awareness framework in [12].*

2.3 Approach

We have been exploring three different approaches or metrics for risk quantification that could be amenable for control synthesis in multi-agent systems:

1. **Risk quantification based on Stochastic MPC.** Building on top of recent results in stochastic MPC [3], we have explored the extension of these results to task allocation in multi-agent systems with temporal logic specifications that are given either as STL or LTL specifications.

2. **Risk quantification based on formal abstractions and approximate stochastic simulation relations.** Together with MPI-SWS, we have shown that this type of risk quantification can deal with uncertainty [1] and with networked or connected multi-agent systems [2]. Additionally, as finite-state abstractions tend to scale poorly for multi-agent systems, we have developed novel efficient representations of the risks. This novel representation leverages low rank tensors and can be used as a local risk measures for a per agent policy synthesis approach [13].

3. **Risk quantification embedded in the propositions of the tasks**. Finally, for multi-agent systems with epistemic uncertainties, we have investigated a novel distribution-based temporal logic that is directly defined over the belief space of the system variables using stochastic expansions [14]. In this manner, we can directly transform the stochastic planning of such an uncertain system into a deterministic problem over the stochastic expansions of the system variables.

In this deliverable, we will focus on the contributions in Approaches 1 and 2, that is, the stochastic MPC approach and the abstraction-based approach. Our work on risk quantification via stochastic expansion (Approach 3) can be found in [14] and is in many ways compatible with the concepts of general MPC approaches for STL problems (Approach 1).

Chapter 3

Risk-Aware Real-Time Task Allocation for Stochastic Multi-Agent Systems: A Stochastic MPC Approach

3.1 Introduction

Multi-agent systems are frequently employed in complex tasks that require close collaborations. Formal specifications, such as signal temporal logic (STL), are increasingly utilized to define complex collaborative task objectives [15]. The collaborative control of multi-agent systems satisfying STL specifications has been studied using mixed-integer programming [16], funnel functions [17], and control barrier functions [18]. Nevertheless, a critical challenge for the practical implementation of these methods is their reliability in uncertain and dynamic environments. For systems subject to stochastic disturbances, it is important to evaluate the impact of these disturbances on the task accomplishment of individual agents so that the associated risks are restricted [19].

Despite recent efforts, a notable gap exists in the literature regarding the control of stochastic multi-agent systems under real-time allocated STL specifications with probabilistic guarantees. Such systems have significant practical value, exemplified by scenarios such as vehicle routing [20]. For example, consider a scenario where two passengers require taxi rides, as illustrated in Fig. 3.1. This naturally renders a task allocation problem by assigning each passenger a taxi. Finding the most efficient assignment is a critical challenge.

Inspired by the example, this work addresses 1) the problem of task allocation (TA) of STL specifications, which aims to determine an efficient agent-specification mapping in a multi-agent setting, and 2) the control synthesis of agent-specification pairs assigned at runtime with probabilistic guarantees. TA necessitates decomposing specifications assigned to multiple collaborative agents into individual agent specifications, requiring the decomposition of STL specifications both among agents [21, 22] and across time [23]. Previous work yields a linear programming problem [24], efficiently solvable using auction algorithms [25]. With decomposed real-time specifications, control synthesis for stochastic multi-agent systems can be addressed through shrinking-horizon model predictive control (MPC) [26], ensuring bounded risks [27]. In previous work, probabilistic reachable

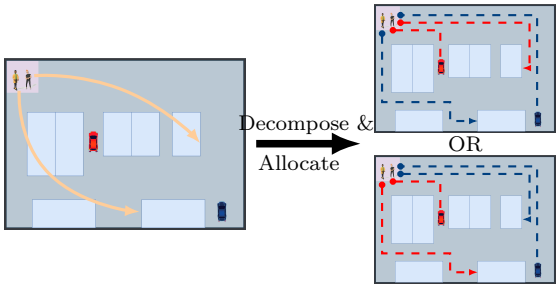


Figure 3.1: Decomposing and allocating tasks for vehicle routing.

tubes have been utilized to optimize risk bounds for both predefined specifications [3] and real-time specifications [28]. These methods underpin the risk-aware control synthesis of stochastic multi-agent systems with decomposed real-time specifications. In [28], we have developed a stochastic MPC algorithm for single-agent settings that can handle specifications that arrive at run-time. In [29], we have proposed a novel approach to allocating STL specifications, involving subgroups of heterogeneous agents and synthesizing control strategies for individual agents in real-time. By decomposing newly assigned specifications into agent-level tasks, we employ a heuristic approach based on STL robustness [30] to assess agent-specification pairs and attack task assignment via an auctioning scheme formulated as a mixed integer linear program.

As trust is crucial in dynamic multi-agent systems, where agents and environment may often change [31], our contribution proposes a novel concept of a trustworthy (correct-by-design) control algorithm for multi-agent systems with real-time task assignment.

In the remainder of this chapter, we will introduce the results of [29] and its connection with the results in [3] and [28]. Additionally, we will connect to the complementary results on MPC with STL in [32], [23], and [14].

3.2 Preliminaries and Problem Statement

For a given probability measure $\mathbb{P}$ defined over a Borel measurable space $(\mathbf{X}, \mathcal{B}(\mathbf{X}))$, we denote the probability of an event $\mathcal{A} \in \mathcal{B}(\mathbf{X})$ as $\mathbb{P}(\mathcal{A})$. In this work, we will work with Euclidean spaces and Borel measurability. Further details on measurability are omitted, and we refer the interested reader to [33]. Additionally, any half-space $H \subset \mathbb{R}^n$ is defined as $H := \{x \in \mathbb{R}^n \mid g^T x \geq b\}$ with $g \in \mathbb{R}^n$ and $b \in \mathbb{R}$. A polyhedron is the intersection of finitely many half-spaces, also denoted as $H := \{x \in \mathbb{R}^n \mid Gx \geq b\}$ with $G \in \mathbb{R}^{q \times n}$ and $b \in \mathbb{R}^q$. A polytope is a bounded polyhedron. The 2-norm is defined by $||x|| = \sqrt{x^T x}$ with $x \in \mathbb{R}^n$. The identity matrix is denoted by $I_n \in \mathbb{R}^{n \times n}$. The vector of all elements one and zero are denoted by $\mathbf{1}_n \in \mathbb{R}^n$ and $\mathbf{0}_n \in \mathbb{R}^n$, respectively. If the context is clear, they will be denoted by $\mathbf{1}$ and $\mathbf{0}$. For any finite set I, the cardinality is denoted by $|I|$. $\text{Proj}_X(Y)$ denotes the projection of elements in Y onto X.

3.2.1 Multi-Agent Systems

We consider a finite ordered set $\mathcal{I}$ of heterogeneous agents where each agent $i \in \mathcal{I}$ has linear time-invariant dynamics with additive noise, given by

$$x^i(k+1) = A^i x^i(k) + B^i u^i(k) + w^i(k), \tag{3.1}$$

where (A^i, B^i) is stabilizable. We impose that all agents have the same state and input dimensions. For the i-th agent, $x^i \in \mathbb{X}^i \subseteq \mathbb{R}^n$ is the state, $x^i(0) \in \mathbb{X}^i$ is an initial state, $u^i \in \mathbb{U}^i \subseteq \mathbb{R}^m$ is the input and $w^i \in \mathbb{R}^n$ is an independent, identically distributed (i.i.d.) noise disturbance with distribution $\mathcal{Q}^i_w$, i.e., $w^i(k) \sim \mathcal{Q}^i_w$, which can have infinite support. We will assume that the distribution has at least a known mean and variance, with the latter required to be strictly positive definite. Additionally, we impose that the distribution is central convex unimodal[1]. Finally, we define that any two disturbances $w^i(k)$ and $w^j(l)$ are independent.

For each agent $i \in \mathcal{I}$, we require that there is a local controller. We define the local controllers as a sequence of policies

$$\boldsymbol{f}^i := \{f^i_0, f^i_1, \dots\},$$

such that $f^i_k{:}\mathbb{H}^i_k{\to}\mathbb{U}^i$ maps the history of states and inputs to inputs. Here $\mathbb{H}^i_k := (\mathbb{X}^i \times \mathbb{U}^i)^k \times \mathbb{X}^i$ with elements $\eta^i(k) := (x^i(0), u^i(0), \cdots, u^i(k-1), x^i(k))$. By implementing controller $\boldsymbol{f}^i$ upon the corresponding agent, we obtain the controlled form of (3.1) for which the control input satisfies the feedback law $u^i(k) = f^i_k(\eta^i(k))$ with $\eta^i(k) \in \mathbb{H}^i_k$. We indicate the input sequence of agent i by $\boldsymbol{u}^i := \{u^i(0), u^i(1), \dots\}$ and define its executions as sequences of states $\boldsymbol{x}^i := \{x^i(0), x^i(1), \dots\}$ referred to as signals.

We define the suffix, segment and signal element of any signal $\boldsymbol{x}^i := \{x^i(0), x^i(1), \dots\}$, respectively, by $\boldsymbol{x}^i_k = \{x^i(k), x^i(k+1), \dots\}$, $\boldsymbol{x}^i_{[a,b]} := \{x^i(a), \dots, x^i(b)\}$ with $a \leq b$, and $\boldsymbol{x}^i(k) := \boldsymbol{x}^i_{[k,k]}$. We define the concatenation of $\boldsymbol{x}^i$, for all agents $i \in I \subseteq \mathcal{I}$, by a higher dimensional signal $\boldsymbol{x}^I$, where $\boldsymbol{x}^I(k) := [\boldsymbol{x}^{i_1}(k)^T, \cdots, \boldsymbol{x}^{i_l}(k)^T]^T$, $i_j \in I$, and $l = |I|$ is the number of elements in I. Any signal $\boldsymbol{x}^i$ can be interpreted as a realization of the probability distribution induced by implementing controller $\boldsymbol{f}^i$, denoted by $\boldsymbol{x}^i \sim \mathbb{P}_{\boldsymbol{f}^i}$. Similar realizations exist for segments, i.e., $\boldsymbol{x}^i_k \sim \mathbb{P}_{\boldsymbol{f}^i,k}$ and concatenations, i.e., $\boldsymbol{x}^I \sim \mathbb{P}_{\boldsymbol{f}^I}$, with $\boldsymbol{f}^I := \{\boldsymbol{f}^i \mid i \in I\}$.

3.2.2 Signal Temporal Logic & Probabilistic Satisfaction

To mathematically describe tasks, e.g., reaching a target within a given time frame, we consider specifications given by signal temporal logic (STL). Here, we assume that all STL specifications adhere to the *negation normal form* (NNF), see [15]. This assumption does not restrict the overall framework since [35] shows that every STL specification can be rewritten into the negation normal form.

STL consists of predicates μ^l for subgroups of l agents that are either true ($\top$) or false ($\bot$). Each predicate μ^l is described by a function $h : \mathbb{R}^{nl} \to \mathbb{R}^q$, $l \leq |\mathcal{I}|$, as follows

$$\mu^l := \begin{cases} \top \text{ if } \forall i \in \{1, \cdots, q\},\ h_i(x) \geq 0 \\ \bot \text{ if } \exists i \in \{1, \cdots, q\},\ h_i(x) < 0. \end{cases} \tag{3.2}$$

[1] $\mathcal{Q}^i_w$ is in the closed convex hull of all uniform distributions on symmetric compact convex bodies in $\mathbb{R}^n$ (c.f. [34, Def. 3.1]).

Here $h_i : \mathbb{R}^{nl} \to \mathbb{R}$ indicates the i-th element of function h. We will assume that all predicate functions are affine functions of the form $h(x) = Gx - b$ with $G \in \mathbb{R}^{q \times nl}$, $b \in \mathbb{R}^q$ and $||g_j|| = 1$ for all $j \in \{1, \cdots, q\}$, where g_j is the j-th row of G. Furthermore, we require that $H_{\mu^l} := \{x \in \mathbb{R}^{nl} \mid Gx \geq b\}$ describes a polytope. Let the set of all predicates be given by $\mathcal{P}$. The STL syntax will be given by

$$\phi ::= \top \mid \mu^l \mid \neg\mu^l \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \Box_{[a,b]}\phi \mid \phi_1 \; U_{[a,b]} \; \phi_2,$$

where $\mu^l \in \mathcal{P}$, ϕ, ϕ_1 and ϕ_2 are STL formula, $a, b \in \mathbb{N}$, and $a \leq b$. The semantics are given next, where $\boldsymbol{x}_k^{\mathcal{I}} \vDash \phi$ denotes the satisfaction of ϕ verified over the suffix of signal $\boldsymbol{x}^{\mathcal{I}}$.

Definition 4. *The STL semantics, for a specification jointly assigned to a set of agents, are recursively given by:*

$$\begin{aligned}
\boldsymbol{x}_k^{\mathcal{I}} &\vDash \mu^l && \iff \exists I \subseteq \mathcal{I} : \; |I| = l, \; h(\boldsymbol{x}^I(k)) \geq \mathbf{0} \\
\boldsymbol{x}_k^{\mathcal{I}} &\vDash \neg\mu^l && \iff \nexists I \subseteq \mathcal{I} : \; |I| = l, \; h(\boldsymbol{x}^I(k)) \geq \mathbf{0} \\
\boldsymbol{x}_k^{\mathcal{I}} &\vDash \phi_1 \wedge \phi_2 && \iff \boldsymbol{x}_k^{\mathcal{I}} \vDash \phi_1 \; \textit{and} \; \boldsymbol{x}_k^{\mathcal{I}} \vDash \phi_2 \\
\boldsymbol{x}_k^{\mathcal{I}} &\vDash \phi_1 \vee \phi_2 && \iff \boldsymbol{x}_k^{\mathcal{I}} \vDash \phi_1 \; \textit{or} \; \boldsymbol{x}_k^{\mathcal{I}} \vDash \phi_2 \\
\boldsymbol{x}_k^{\mathcal{I}} &\vDash \Box_{[a,b]} \; \phi && \iff \forall k' \in [k+a, k+b] : \boldsymbol{x}_{k'}^{\mathcal{I}} \vDash \phi \\
\boldsymbol{x}_k^{\mathcal{I}} &\vDash \phi_1 \; U_{[a,b]} \; \phi_2 && \iff \exists k_1 \in [k+a, k+b] : \boldsymbol{x}_{k_1}^{\mathcal{I}} \vDash \phi_2 \\
& && \quad \textit{and} \; \forall k_2 \in [k, k_1], \; \boldsymbol{x}_{k_2}^{\mathcal{I}} \vDash \phi_1.
\end{aligned}$$

Here, we introduced a slight abuse of the notation as $[a, b]$ is used to describe the set of all natural numbers within the real interval $[a, b]$. Note that the definition is invariant of the actual indices of the l agents. Additional operators can be derived such as the eventually-operator $\Diamond_{[a,b]}\phi := \top U_{[a,b]}\phi$.

Since all agents behave stochastically, each STL specification can only be satisfied probabilistically. Should a specification be assigned at time k, the probability can be determined based on the state measurement $x^{\mathcal{I}}(k)$, the controller $\boldsymbol{f}^{\mathcal{I}}$, and the system dynamics (3.1). Accordingly, we consider the probability that suffix fragment $\boldsymbol{x}_k^{\mathcal{I}}$ satisfies specification ϕ, given state $x^{\mathcal{I}}(k)$. We denote this by

$$\mathbb{P}_{\boldsymbol{f}}(\phi, k) := \mathbb{P}_{\boldsymbol{f}^{\mathcal{I}},k}(\boldsymbol{x}_k^{\mathcal{I}} \vDash \phi \mid x^{\mathcal{I}}(k)). \tag{3.3}$$

For each specification ϕ, we require that the maximal allowable risk $r_{\phi,\max}$ of not satisfying specification ϕ is given. In the remainder, we will refer to this as the maximal risk. Note that for $|\mathcal{I}| = 1$, the above definitions of STL and probability will become the standard ones as seen in [28].

3.2.3 Problem Statement & Approach

Consider each agent $i \in \mathcal{I}$ has dynamics (3.1) with noise distribution $\mathcal{Q}_w^i$ and state update $x^i(k)$, at each time step $k \in \mathbb{N}$. We assume that a new specification may be provided at any time. The objective is to update the suffix controller $\boldsymbol{f}_k^{\mathcal{I}}$ at each time step $k \in \mathbb{N}$. Here, any newly provided specification is either accepted or rejected based on the maximal risks of the newly provided specification and previously accepted specifications.

Problem Statement. Given a sequence of STL specifications $\boldsymbol{\phi} := \{\phi_1, \cdots, \phi_t\}$ assigned, respectively, at time instances $\{k_1, \cdots, k_t\}$ with $k_i < k_j$ for $i < j$, with maximal risk $r_{\phi_i,\max}$, develop a method for *updating*, at each time step $k \in \mathbb{N}$, suffix $\boldsymbol{f}_k^{\mathcal{I}}$ such that $\mathbb{P}_{\boldsymbol{f}}(\phi_i, k_i) \geq 1 - r_{\phi_i,\max}$ if $k_i \leq k$ and ϕ_i is a previously accepted specification, where ϕ_i with $k_i = k$ is allowed to be rejected.

Approach. First, a newly provided STL specification and maximal risk are decomposed onto the single-agent level (Section 3.3.2). Subsequently, a filter determines which agent-specification pairs are favorable based on STL robustness and heuristics (Section 3.3.3). Afterwards, for each remaining agent-specification pair, a potential control strategy is determined together with the local risk value (Section 3.3.4). Based on the local risk value for each feasible agent-specification pair, auctioning determines the definitive assignment (Section 3.3.5). Should no valid assignment exist, the specification is rejected. Finally, the control strategy for each agent is updated and implemented (Section 3.3.6).

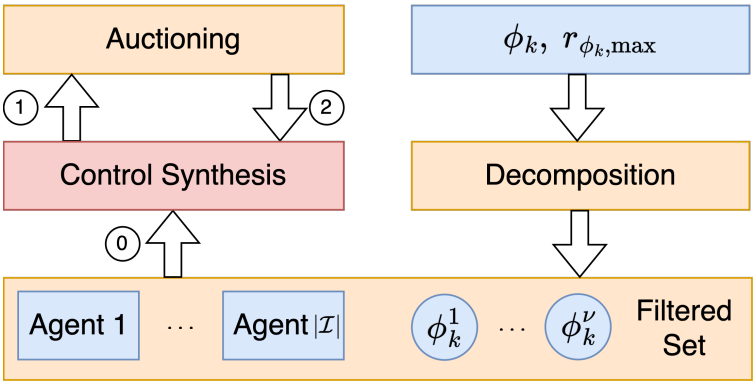


Figure 3.2: Illustration of the approach at time k: (0) Determine for each agent-specification pair a control strategy and the local risk value. (1) Auction the offers from agent-specification pairs. (2) Make a decision based on the auctioning outcome.

3.3 Risk-Aware Multi-Agent Task Allocation and Control Synthesis

To tackle the problem above, we will first consider agent dynamic decomposition, specification decomposition and heuristic agent-specification filtering. Second, we establish control synthesis and local risk value computation for any individual agent-specification pair. Third, we utilize auctioning, based on the local risk values, to obtain a definitive agent-specification assignment. This section concludes with an implementation algorithm and theoretical analysis of the overall method.

3.3.1 Decomposing Agent Dynamics

We decompose the dynamics (3.1), given by

$$x^i(k+1) = A^i x^i(k) + B^i u^i(k) + w^i(k),$$

into a nominal and an error part [36]. The nominal dynamics, denoted as z^i, contain no stochasticity, and the (stochastic) error dynamics, denoted as e^i, are autonomous. This yields

$$z^i(k+1) = A^i z^i(k) + B^i v^i(k), \tag{3.4a}$$

$$e^i(k+1) = A^i_K e^i(k) + w^i(k), \tag{3.4b}$$

$$\text{with } x^i(k) = z^i(k) + e^i(k), \tag{3.4c}$$

$$u^i(k) = v^i(k) + K^i e^i(k). \tag{3.4d}$$

Here $A^i_K = A^i + B^i K^i$ and K^i is a stabilizing feedback gain meant to keep the error e^i small. Throughout this section, we omit superscripts indicating agent indices when appropriate.

3.3.2 Specification Decomposition

Let ϕ be a newly assigned STL specification with maximal risk $r_{\phi,\max}$, involving a distinct number of agents, $\nu \leq |\mathcal{I}|$. The objective is to decompose the specification ϕ into agent-level sub-specifications, ϕ^j, $j \in \mathcal{I}$, and allocate individual maximal risks, $r_{\phi^j,\max}$, in accordance to $r_{\phi,\max}$. First, we define $\text{dec}(\phi, \nu) = \wedge_{j=1}^{\nu} \phi^j$ as the decomposition of ϕ into a conjunctive form, with each conjunct representing a sub-specification, ϕ^j, the satisfaction of which depends on a single agent j, and ν being the number of agents needed to satisfy ϕ. Second, we assign a maximal risk to ϕ^j, following a union-bound argument, which requires $\sum_{j=1}^{\nu} r_{\phi^j,\max} \geq r_{\phi,\max}$. In this work, we select a uniform allocation of risks across sub-specifications, obtaining $r_{\phi^j,\max} = \frac{r_{\phi,\max}}{\nu}$, for all agents $j \in \{1, \ldots, \nu\}$.

The decomposition $\text{dec}(\phi, \nu)$ acts as a transformation on ϕ returning a conjunctive formula, the satisfaction of which implies satisfaction of ϕ. However, the converse is not necessarily true, implying a lack of completeness. There are a few methods [21, 22, 37] for splitting STL specifications into sub-specifications. In this work, given a specification ϕ, we convert Boolean and temporal operators to conjunctive forms based on the heuristics in [37] and decompose predicates following a similar approach to [21]. Therein, predicates are defined on general convex sets, whereas negated forms are not studied. Here, we focus on polytopic settings and allow negations. For our setting, we assume the following.

Assumption 1. *Let ϕ be an STL specification assigned at time k. There exists decomposition $\text{dec}(\phi, \nu) = \wedge_{j=1}^{\nu} \phi^j$, where ϕ^j indicates a task involving a single agent.*

In view of existing decomposition approaches, where sub-specifications are assigned to subteams of agents, Assumption 1 is restrictive. However, in the setting of polytopic predicates, we can produce agent-level sub-specifications by projecting the associated predicates onto the agents' state space. Before explaining this, we define the following.

Definition 5. *Let $H := \{x \mid h(x) \geq \mathbf{0}\}$ be a polytope in $\mathbb{R}^n$. $B_{\text{in}}(H) := \{x \mid \underline{b}_j^{\text{in}} \leq x_j \leq \overline{b}_j^{\text{in}},\ j = 1, \ldots, n\}$ is an orthotope inscribed in H, i.e., $B_{\text{in}}(H) \subseteq H$, and $B_{\text{out}}(H) := \{x \mid \underline{b}_j^{\text{out}} \leq x_j \leq \overline{b}_j^{\text{out}},\ j = 1, \ldots, n\}$ is an orthotope circumscribing H, i.e., $B_{\text{out}}(H) \supseteq H$.*

The following lemma shows that any (negated) predicate can be decomposed into a conjunction of agent-level sub-predicates, satisfaction of which implies the satisfaction of the original (negated) predicate. The decomposition of the Boolean and temporal operators is described in [37].

Lemma 1. *Let $\mu := (h(x) \geq \mathbf{0})$ be a predicate, $H := \{x \mid h(x) \geq \mathbf{0}\}$ be a polytope, with $x = [x_1^T \ \dots \ x_\nu^T]^T$, $x_i \in \mathbb{X}^i$, $i \in \{1, \dots, \nu\}$, and $B_{\text{in}}(H) \subseteq H$ and $B_{\text{out}}(H) \supseteq H$ be the associated orthotopes inscribed in and circumscribing H, respectively. Then, it holds that (i) $\text{dec}(\mu, \nu) = \wedge_{i=1}^{\nu} \mu_{\text{in}}^i$ satisfies μ, with $\mu_{\text{in}}^i := (x_i \in \text{Proj}_{\mathbb{X}^i}(B_{\text{in}}(H))$, and (ii) $\text{dec}(\neg\mu, \nu) = \wedge_{i=1}^{\nu} \neg\mu_{\text{out}}^i$ satisfies $\neg\mu$, with $\mu_{\text{out}}^i := (x_i \in \text{Proj}_{\mathbb{X}^i}(B_{\text{out}}(H))$.*

Proof. (i) Let $\mu_{\text{in}} := (x \in B_{\text{in}}(H))$. By construction, it holds that $B_{\text{in}}(H) \equiv \text{Proj}_{\mathbb{X}^1}(B_{\text{in}}(H)) \times \dots \times \text{Proj}_{\mathbb{X}^\nu}(B_{\text{in}}(H))$, implying that $\mu_{\text{in}} \equiv \wedge_{i=1}^{\nu} \mu_{\text{in}}^i$. Since, $B_{\text{in}}(H) \subseteq H$, μ_{in} satisfies μ, which leads to $\wedge_{i=1}^{\nu} \mu_{\text{in}}^i$ satisfying μ as needed. (ii) Let $\mu_{\text{out}} := (x \in B_{\text{out}}(H))$. Similarly to (i), it holds that $\mu_{\text{out}} \equiv \wedge_{i=1}^{\nu} \mu_{\text{out}}^i$, since $B_{\text{out}}(H) \equiv \text{Proj}_{\mathbb{X}^1}(B_{\text{out}}(H)) \times \dots \times \text{Proj}_{\mathbb{X}^\nu}(B_{\text{out}}(H))$, by construction. Then, we have $\neg\mu_{\text{out}} \equiv \vee_{i=1}^{\nu} \neg\mu_{\text{out}}^i$. Since $\wedge_{i=1}^{\nu} \neg\mu_{\text{out}}^i$ satisfies $\vee_{i=1}^{\nu} \neg\mu_{\text{out}}^i$ and $\neg\mu_{\text{out}}$ satisfies $\neg\mu$, we have that $\wedge_{i=1}^{\nu} \neg\mu_{\text{out}}^i$ satisfies $\neg\mu$, completing the proof. □

We consider the following example to illustrate the proposed specification decomposition approach considered in this work (see Fig. 3.3).

Example 1. *Consider formulas $\phi = \Box_{[a,b]}\mu$ and $\psi = \Box_{[a,b]}\neg\mu$, where $\mu := (h(x) \geq \mathbf{0})$ is a predicate, with $h(x) = \begin{bmatrix} c_1 & c_2 \end{bmatrix} x + \mathbf{1}_4$, $c_1 = \begin{bmatrix} -\mathbf{1}_2^T & \mathbf{1}_2^T \end{bmatrix}^T$, $c_2 = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}^T$. We wish to convert ϕ and ψ into conjunctive forms of two conjuncts (subformulas). First, we define the polytope $H = \{x \mid h(x) \geq \mathbf{0}\}$, and construct orthotopes $B_{\text{in}}(H) = \{x \mid b_{\text{in}}(x) \geq \mathbf{0}\}$ and $B_{\text{out}}(H) = \{x \mid b_{\text{out}}(x) \geq \mathbf{0}\}$, with $b_{\text{in}}(x) = 2 \begin{bmatrix} \boldsymbol{I}_2 & -\boldsymbol{I}_2 \end{bmatrix}^T x + \mathbf{1}_4$ and $b_{\text{out}}(x) = \begin{bmatrix} \boldsymbol{I}_2 & -\boldsymbol{I}_2 \end{bmatrix}^T x + \mathbf{1}_4$, respectively, as in Definition 5. Polytope H and orthotopes $B_{\text{in}}(H)$, $B_{\text{out}}(H)$ are depicted in Fig. 3.3. Then, using Lemma 1, we define predicates $\mu_{\text{in}}^i := (x_i \in \text{Proj}_{\mathbb{X}^i}(B_{\text{in}}(H)) = (-0.5 \leq x_i \leq 0.5)$, $\mu_{\text{out}}^i := (x_i \in \text{Proj}_{\mathbb{X}^i}(B_{\text{out}}(H)) = (-1 \leq x_i \leq 1)$, with $i \in \{1, 2\}$, allowing us to decompose μ and $\neg\mu$ as $\text{dec}(\mu, 2) = \mu_{\text{in}}^1 \wedge \mu_{\text{in}}^2$ and $\text{dec}(\neg\mu, 2) = \neg\mu_{\text{out}}^1 \wedge \neg\mu_{\text{out}}^2$. Finally, $\text{dec}(\phi, 2) = \phi^1 \wedge \phi^2$ and $\text{dec}(\psi, 2) = \psi^1 \wedge \psi^2$, where $\phi^i = \Box_{[a,b]}\mu_{\text{in}}^i$ and $\psi^i = \Box_{[a,b]}\neg\mu_{\text{out}}^i$, with $i \in \{1, 2\}$, are valid decompositions of ϕ and ψ, respectively.*

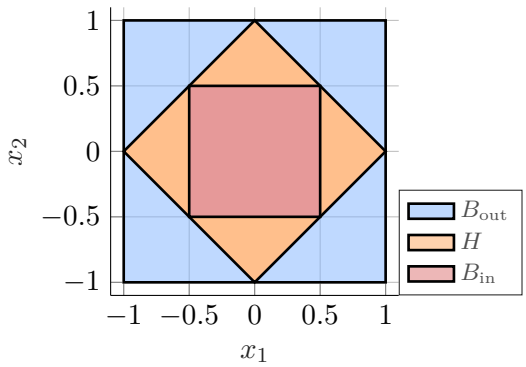


Figure 3.3: Polytopes H, $B_{\text{in}}(H)$, $B_{\text{out}}(H)$, for Example 1

3.3.3 Heuristic Agent-Specification Filtering

STL is equipped with robustness metrics for assessing the robust satisfaction of a formula [30]. A scalar-valued function $\rho^\phi(\boldsymbol{x}, k)$ of a signal $\boldsymbol{x}$ and time k indicates how robustly a signal $\boldsymbol{x}$ satisfies

a formula ϕ at time k. The robustness function is defined recursively in [30, Definition 3].

Let ϕ_k be a newly assigned STL specification at time k, and assume that $\text{dec}(\phi_k, \nu_k) = \wedge_{j=1}^{\nu_k} \phi_k^j$. In the following, we require that the total number of agents, denoted by $|\mathcal{I}|$, satisfies $|\mathcal{I}| > \nu_k$, otherwise, the following filtering procedure is ignored. Although, ϕ_k is split into ν_k agent-level sub-specifications, the assignment of ϕ_k^j to a specific agent $i \in \mathcal{I}$ is not determined by $\text{dec}(\phi_k, \nu_k)$. In fact, each sub-specification, ϕ_k^j, may be assigned to any agent $i \in \mathcal{I}$ resulting in $\nu_k|\mathcal{I}|$ agent-specification pairs. To reduce the number of potential pairs, we utilize the following heuristic approach: Based on the quantitative semantics of STL [30], we compute the robustness function of ϕ_k^j over the ith agent's trajectory. This metric indicates roughly the distance between the trajectory of agent i and the space of signals that satisfy ϕ_k^j [38], thereby facilitating a preliminary agent-specification assessment. We approximate the ith agent's trajectory by $\boldsymbol{z}^{i,k-1} = \{z_0^{i,k-1}, z_1^{i,k-1}, \ldots\}$, $i \in \mathcal{I}$, that is, the signal associated with the (deterministic) nominal dynamics of agent i, (3.4a), computed at time $k-1$, which is available at time k. Specifically, for each $j \in \{1, \ldots, \nu_k\}$, and $i \in \mathcal{I}$, we efficiently calculate $\rho^{\phi_k^j}(\boldsymbol{z}^{i,k-1}, k)$, which requires negligible computational cost. Subsequently, for each ϕ_k^j, we collect the largest ν_k values of $\rho^{\phi_k^j}(\boldsymbol{z}^{i,k-1}, k)$ among the $|\mathcal{I}|$ options, resulting in $(\nu_k)^2$ agent-specification pairs. Hence, should $\nu_k \ll |\mathcal{I}|$, the filtering significantly reduces potential pairs.

3.3.4 Control Synthesis and Local Risk Value

In this subsection, we compute the local risk value and synthesize a controller for a given agent-specification pair utilizing previous work on risk-aware MPC for single-agent systems [28]. We refer the reader to [28] for more details.

We assume a new specification ϕ is provided at time k together with maximal risk $r_{\phi,\max}$. For any specifications ψ, we denote with k_ψ the time at which the specification was assigned. We denote with $\mathcal{P}^k$ the set of all previously accepted specifications ψ with $k_\psi < k$, together with the newly assigned specification ϕ, i.e., $\phi \in \mathcal{P}^k$. The objective is to synthesize a controller $\boldsymbol{f}$ such that $\varphi := \wedge_{\psi \in \mathcal{P}^k} \Diamond_{[k_\psi, k_\psi]} \psi$ is satisfied in accordance to the maximal risk of each conjunction element, i.e., $\mathbb{P}_{\boldsymbol{f}}(\psi, k_\psi) \geq 1 - r_{\psi,\max}$, $\forall \psi \in \mathcal{P}^k$. To accomplish the objective, we make the following assumption.

Assumption 2. *Distribution $\mathcal{Q}_w$ has zero mean and $A_K \Sigma_\infty A_K^T + var(\mathcal{Q}_w) = \Sigma_\infty$ has solution $\Sigma_\infty = I_n$*

Note that the above assumption is not restrictive as a simple coordinate transformation $y = \Sigma_\infty^{-\frac{1}{2}} x$ will ensure the second part of the assumption is satisfied. We call the transformed dynamics the *normalized dynamics*, and in the remainder, we maintain our notations as if the dynamics are normalized.

We are interested in implementing a tube-based MPC algorithm that at each time instance $k \in [0, N]$ recomputes the optimal nominal trajectory $\boldsymbol{z}_{[k,N]}$, the optimal nominal input $\boldsymbol{v}_{[k,N-1]}$, and the optimal risk levels $\boldsymbol{r}_{[k,N]}$. For this, we assume given $x(k)$ together with $\boldsymbol{z}_{[0,k]}$, $\boldsymbol{v}_{[0,k-1]}$, $\boldsymbol{\rho}_{[0,k]}$ and $\boldsymbol{r}_{[0,k]}$ computed at time $k-1$. Furthermore, for all $\psi \in \mathcal{P}^k$, we let $\mathcal{H}_\psi^{k_\psi} \subseteq [0, N]$ without loss of generality. Here, the active horizon $\mathcal{H}_\phi^k$ is the set of all time instances needed to evaluate $\boldsymbol{x}_k \vDash \phi$ (see [28, Definition 4]). It should be noted that any active horizon consists of only finite elements.

Let us consider the following tube-based MPC problem

$$\min_{\Xi} \; J(\boldsymbol{z}_{[0,N]}, \boldsymbol{v}_{[0,N-1]}, \boldsymbol{r}_{[0,N]}), \tag{3.5a}$$

$$\text{s.t.} \quad z(k) = x(k) \tag{3.5b}$$

$$z(\tau+1) = Az(\tau) + Bv(\tau), \forall \tau \in \{k, \cdots, N-1\}, \tag{3.5c}$$

$$G(j)z(j) + MF(j)s(k) \geq b(M,j) + \rho(j)\mathbf{1}, \tag{3.5d}$$

$$Cs(k) \geq d, \; \epsilon \leq \rho(j) \leq \tfrac{M}{2}, \; \forall j \in \{0, \cdots, N\}, \tag{3.5e}$$

$$n = \rho(j)^2 r(j), \qquad \forall j \in \{0, \cdots, N\}, \tag{3.5f}$$

$$r_{\psi,\max} \geq \textstyle\sum_{j \in \mathcal{H}_{\psi}^{k_\psi} \setminus \{k_\psi\}} r(j), \forall \psi \in \mathcal{P}^k, \tag{3.5g}$$

where $\Xi := \{\boldsymbol{z}_{[k,N]}, \boldsymbol{v}_{[k,N-1]}, \boldsymbol{r}_{[k+1,N]}, \boldsymbol{\rho}_{[k+1,N]}, s(k)\}$, J is either a linear or quadratic cost function, the constraints (3.5d)-(3.5e) are obtained from $\varphi = \wedge_{\psi \in \mathcal{P}^k} \Diamond_{[k_\psi, k_\psi]} \psi$ and the constraints (3.5f)-(3.5g) are obtained from [28, Thm. 4]. From the above tube-based MPC, an (updated) control strategy is obtained by $\boldsymbol{v}_{[k,N-1]}$, and a local risk value $r_{\phi,mpc}$ is obtained by the right-hand side of (3.5g), i.e.,

$$r_{\phi,mpc} = \textstyle\sum_{j \in \mathcal{H}_{\phi}^{k} \setminus \{k\}} r(j). \tag{3.6}$$

A solution to (3.5), by [28, Thm. 4], ensures that for all $\psi \in \mathcal{P}^k$, $\mathbb{P}_{\boldsymbol{f}}(\psi, k_\psi) \geq 1 - r_{\psi,\max}$. Should there exist no solution to problem (3.5), the newly provided agent-specification pair is deemed infeasible. In such a case, we will update the control strategy based only on measurement $x(k)$, as per constraint (3.5b). This can be achieved by considering the MPC problem (3.5) with $\mathcal{P}^k = \mathcal{P}^k \setminus \{\phi\}$. However, should the measurement also fail to produce a controller, the previously computed control strategy will be implemented instead.

3.3.5 Task Allocation Algorithm

Let ϕ be a specification which can be decomposed into ν sub-specifications. After filtering agent-specification pairs and attempting optimization problem (3.5) with the remainder, a list $\mathbb{L} \subset \mathcal{I} \times \{1, \cdots, \nu\}$ of feasible agent-specification pairs is generated. Each feasible agent-specification pair has a local risk value (3.6) denoted by $r^i_{\phi_j,mpc}$, with $i \in \mathcal{I}$ and $j \in \{1, \cdots, \nu\}$. Our objective is to auction sub-specifications to agents based on the local risk values, obtaining a definitive assignment in the process. Here, each agent can only receive one specification, and all specifications must be allocated.

To determine an agent-specification assignment, we solve the following binary optimization problem given by

$$\min_{\boldsymbol{\lambda}} \; \textstyle\sum_{(i,j) \in \mathbb{L}} \lambda_{i,j} r^i_{\phi_j,mpc}, \tag{3.7a}$$

$$\text{s.t.} \; \textstyle\sum_{i \in \mathcal{I}} \lambda_{i,j} = 1, \; \forall j \in \{1, \cdots, \nu\} \tag{3.7b}$$

$$\textstyle\sum_{j \in \{1, \cdots, \nu\}} \lambda_{i,j} \leq 1, \; \forall i \in \mathcal{I}, \tag{3.7c}$$

$$\lambda_{i,j} = 0, \qquad \forall (i,j) \notin \mathbb{L}, \tag{3.7d}$$

$$\lambda_{i,j} \in \{0,1\}, \; \forall (i,j) \in \mathbb{L}, \tag{3.7e}$$

where $\boldsymbol{\lambda} = \{\lambda_{i,j} \mid (i,j) \in \mathbb{L}\}$. Here, constraint (3.7b) entails that each specification must be assigned to exactly one agent; constraint (3.7c) entails that each agent can at most receive one specification; and $\lambda_{i,j} = 1$ implies agent i and sub-specification j are chosen as an agent-specification pair. The objective is to minimize the sum of the local risk values, thereby ensuring the best probability satisfaction of specification ϕ. Should no solution be found, specification ϕ will be rejected.

3.3.6 Implementation & Theoretical Analysis

To enact the proposed method, we consider Algorithm 1, which demonstrates multi-agent task allocation and control strategy updates for each agent at every time step. For a more detailed algorithm on single-agent control synthesis, we defer to [28] and the algorithm therein.

Algorithm 1 Task Allocation and Control Strategy Update

1: Given: Normalized system (3.1) with $x^i(0),\ \forall i \in \mathcal{I}$
2: **for** $k \in \{0, \cdots, N-1\}$ **do**
3: **if** no new spec. **then** set $\mathbb{L}_{opt} = \emptyset$ & <u>go to line 15</u>
4: *Task Allocation:*
5: $(\psi,\ r_{\psi,\max}) \leftarrow$ load new specification
6: Decompose into $(\psi^j,\ r_{\psi^j,\max})$, $j \in \{1, \cdots, \nu\}$
7: Filter pairs to obtain $\mathbb{L} \subset \mathcal{I} \times \{1, \cdots, \nu\}$
8: **for** $(i,j) \in \mathbb{L}$ **do**
9: Solve (3.5), get $r^i_{\psi^j,mpc}$ and control strategy $\boldsymbol{f}^{(i,j)}$
10: **if** no solution **then** $\mathbb{L} = \mathbb{L} \setminus \{(i,j)\}$
11: **end for**
12: Solve (3.7) to get definitive assignment list $\mathbb{L}_{opt}$
13: *Control Strategy Update:*
14: $\forall (i,j) \in \mathbb{L}_{opt}$, set $\boldsymbol{f}^{i,k} = \boldsymbol{f}^{(i,j)} \leftarrow$ update
15: **for** $i \in \mathcal{I} \setminus \mathrm{Proj}_{\mathcal{I}}(\mathbb{L}_{opt})$ **do**
16: Solve (3.5) to get control strategy $\boldsymbol{f}^i_{meas}$
17: **if** solution **then** set $\boldsymbol{f}^{i,k} = \boldsymbol{f}^i_{meas} \leftarrow$ update
18: **else** set $\boldsymbol{f}^{i,k} = \boldsymbol{f}^{i,k-1} \leftarrow$ no update
19: **end for**
20: Implement $\boldsymbol{f}^{\mathcal{I},k}$ and measure $x^i(k+1),\ \forall i \in \mathcal{I}$
21: **end for**

Let us consider the recursive feasibility of updating a control strategy for each agent at any time.

Theorem 1 (Recursive feasibility)**.** *If there exists a feasible control strategy $\boldsymbol{f}^{\mathcal{I},k}$, at time k, then there exists a feasible control strategy $\boldsymbol{f}^{\mathcal{I},k+1}$ at time $k+1$.*

Proof. It is sufficient to prove that the existence of $\boldsymbol{f}^{i,k}$ implies the existence of $\boldsymbol{f}^{i,k+1}$ for each agent $i \in \mathcal{I}$. Notice that if an agent either accepts a new sub-specification or measurement only, an updated control strategy is obtained, respectively, as $\boldsymbol{f}^{i,k+1} = \boldsymbol{f}^{(i,j)}$ or $\boldsymbol{f}^{i,k+1} = \boldsymbol{f}^i_{meas}$. If neither is available, the previous control strategy is again implemented, $\boldsymbol{f}^{i,k+1} = \boldsymbol{f}^{i,k}$, completing the proof. □

For each specifications ϕ accepted at time k, by subgroup $I \subseteq \mathcal{I}$, we need that $\mathbb{P}_{\boldsymbol{f}}(\phi, k) \geq 1 - r_{\phi,\max}$. Here, it is sufficient to show that $\forall i \in I$, $\mathbb{P}_{\boldsymbol{f}}(\phi^i, k) \geq 1 - r_{\phi^i,\max}$. This, in relation to optimization problem (3.5), is discussed within open-loop and closed-loop implementation in our previous work [28]. Therein, we established the validity of open-loop implementation and suggested that closed-loop implementation follows naturally from the convex unimodality of the disturbance.

Remark 2. *Note that the local risk value $r_{\phi^j,mpc}$ compared to the average maximal risk $r_{\phi^j,\max}$ is conservative, $r_{\phi^j,mpc} \geq r_{\phi^j,\max}$, due to a union-bound argument used in proving [28, Thm. 4]. Also, the obtained task allocation may not always be optimal or feasible due to our heuristic choice of filtering, even if an allocation exists.*

3.4 Case Study

We validate our proposed method using a multi-shuttle routing case. In a tourist attraction spot (Fig. 3.4), four shuttle buses of differing speeds (A-D) should pick up tourists from gathering points (GP) and deliver them to an unloading point (ULP) while avoiding two buildings (B). The shuttles initiate from their respective terminals (T) and should return when required. Each task should be assigned to a most probable shuttle to fetch the tourists. Thus, new routing may be assigned anytime, rendering a dynamic task allocation problem.

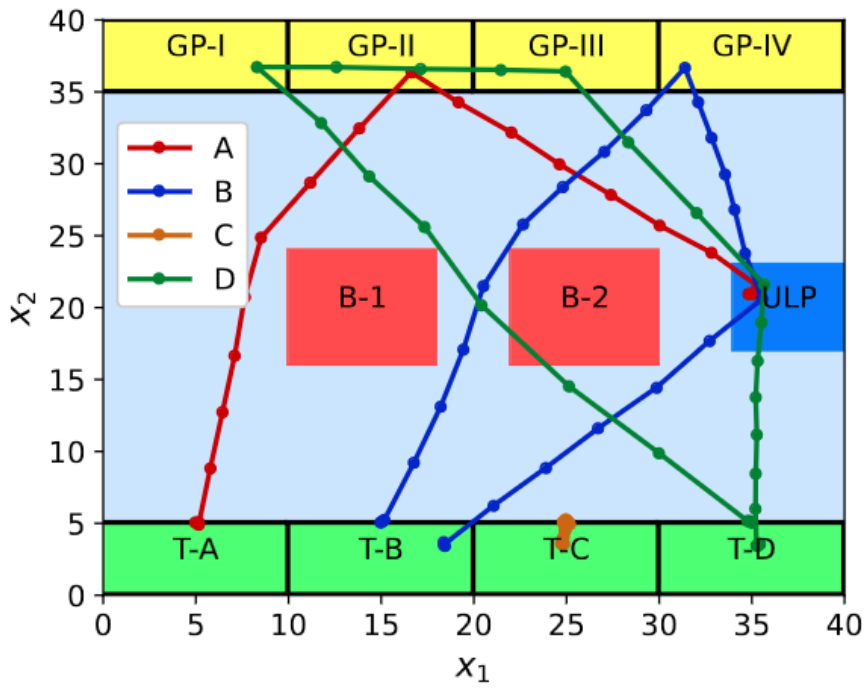


Figure 3.4: The shuttle trajectories.

After performing the decomposition of specifications, the pre-filtering procedure, synthesizing the corresponding control strategies, and conducting the auction mechanism among agentâ€“specification pairs, we obtain the resulting allocation and system evolution illustrated in Fig. 3.4 The logging information of the experiment indicates that: 1) at time $k = 2$, buses 'A', 'B', and 'D' are assigned

with GP-II, GP-IV, and GP-I, respectively; 2) at time $k = 8$, buses 'A' and 'D' are assigned with GP-II and GP-III, respectively; 3) at time $k = 15$, buses 'B', 'C', and 'D' have accepted the return specification, while shuttle 'A' has rejected it. Bus 'C' is not assigned any task throughout the experiment and, as a result, stays at its terminal. Since buses 'A' and 'D' accept additional tasks at $k = 8$, our method allows agents to adjust their behaviours according to newly assigned tasks. As mentioned, shuttle 'A' has rejected the return task at time $k = 15$. This is due to the incapability to move long distances considering its small speed limit. Additionally, all buses are confined within the 'BOX' region and kept outside the buildings, implying the compatibility of our method to explicitly assigned specifications. Part of the trajectory of shuttle 'D' intersects with B-2 due to naive linear interpolation. In practice, nonlinear interpolation can guarantee critical safety, which may generate curved trajectories when navigating around the building.

Remark 3. *In this experiment, the computation of control inputs by solving problem* (3.5) *is distributed to each agent, leading to a computational complexity linear to the number of assigned specifications. In this sense, our proposed method allows for a comparable computational load of each agent to a conventional single-agent case, allowing it to be implemented in practical scenarios.*

3.5 Connection to Complementary Work and Conclusions

In this chapter, we have considered real-time task allocation and control synthesis for stochastic heterogeneous linear multi-agent systems with probabilistic satisfaction of STL specifications. The proposed method relies on the dynamic allocation of decomposed specifications to individual agents using risk measures via auctioning and on the stochastic MPC for run-time specifications [28]. More precisely, the risk measures are computed using tube-based MPC on the agent level during the control synthesis as in [28]. Additionally, the following two contributions on splitting specifications in time and using direct-data driven MPC for STL yield complementary results.

3.5.1 Splitting Specifications in Time

An important aspect of risk quantification and mitigation in MAS evolves over how tasks and their risks are distributed in time and over the different agents. This has resulted in a complementary result [23] that is summarized below.

The control synthesis of a dynamic system subject to an STL specification is commonly formulated as a MILP/MICP problem. Solving such a problem is computationally expensive when the specification is long and complex. Therefore, we have proposed a framework to transform a long and complex specification into separate forms in time, to be more specific, the logical combination of a series of short and simple subformulas with non-overlapping timing intervals [23]. In this way, one can easily modularize the synthesis of a long specification by solving its short subformulas, which improves the efficiency of the control problem. We first propose a syntactic timing separation form for a type of complex specifications based on a group of separation principles. Then, we further propose a complete specification split form with subformulas completely separated in time. Based on this, we develop a modularized synthesis algorithm that ensures the soundness of the solution to the original synthesis problem.

3.5.2 Direct-data Driven Approach to MPC

Additionally, for single-agent systems, we have already started working on introducing uncertainty information and dealing with missing information. Complementary work that achieves this has been given in [32] and is summarized next.

To achieve the reliability of safety-critical systems, such as autonomous vehicles and power grids, it is crucial to give formal guarantees on their functionality. Hence, it is beneficial to automatically construct a provably correct controller, where the desired behavior is described by temporal logic specifications. In this work, we focus on STL since it does not only provide a qualitative answer to whether the specification is satisfied but also provides insight into the quality of satisfaction (or violation). Furthermore, it is an extension of LTL with real-time and real-valued constraints.

Methods that synthesize a provably correct controller for a system subject to temporal logic specifications are often referred to as correct-by-design control synthesis methods. Most of them depend on the availability of an analytic model of the system. However, due to the increasing complexity of systems, obtaining an accurate model has become a challenging task. Furthermore, there is always unmodeled behavior due to first-principles-based modeling or uncertainty of the estimated model. A way to circumvent this issue is to directly synthesize a controller from data. By doing so, we avoid the possibility that some form of approximation error occurs in the modeling process. In this work, we bring down correct-by-design control synthesis to the level of data by using a data-driven method within the behavioral framework to directly synthesize a controller. More specifically, after collecting a single sequence of input-output data from the system, we construct a data-driven characterization of the system behavior. Using this data-driven characterization we show that we can synthesize a controller, such that the controlled system satisfies an STL specification. The underlying optimization problem is solved by MILP.

3.5.3 Conclusions

In this chapter, we show how (stochastic) MPC can be used to quantify risks while synthesizing control strategies for multi-agent systems. In the next chapter, we will show how abstraction-based solutions can be scaled up to multi-agent systems and how a local risk measure based on a low rank tensor value function can be used for local strategy design.

Chapter 4

Efficient Risk Quantification of Multi-Agent Systems via Tensorized DP and Simulation Relations

4.1 Introduction

Advances in computational power have enabled the development of large-scale systems in safety-critical domains like smart grids and traffic management [39]. These systems involve numerous interacting agents with complex or uncertain dynamics [39, 40]. While connectivity improves coordination, it also increases complexity and risk [41]. This paper addresses the challenge of scalable safety control and verification, beginning with a review of safety control methods, as scalability challenges stem from the underlying control mechanisms themselves. Control Barrier Functions are employed in [42] to enforce that the controlled system remains within a desirable set, which serves as a characterization of safety. Drawing inspiration from Model Predictive Control (MPC), the authors in [43, 44] include risk in the optimization framework. Alongside the aforementioned approaches, there exist formal methods [9, 45], which provide a rigorous framework for reasoning about safety across different levels of abstraction [46]. In particular, temporal logic enables systematic specification and verification, offering a structured way to verify probabilistic safety. In this paper, we adopt a correct-by-design synthesis approach [2] to verify safety in a large-scale probabilistic setting.

Correct-by-design control synthesis for stochastic systems with temporal logic specifications often faces a common challenge: the curse of dimensionality [47]. This refers to the exponential increase in computational cost as system size grows, making synthesis intractable. While several works [48, 49, 2, 47, 50] address this issue, they primarily focus on abstraction scalability rather than the computational efficiency of the synthesis algorithm.

Work has also been done in [51, 52, 53] for general dynamic programming problems and in [54] temporal logic control to mitigate computational complexity by leveraging low-rank tensor approximations. Although computing probabilistic safety guarantees for temporal logic specifications involves value iteration, value functions and the use of tensor approximations in [51, 52, 53, 54] are fundamentally different and not suitable for computing probabilistic safety guarantees. To the best of our knowledge, no existing methods focus on low-rank tensor computations for prob-

abilistic correct-by-design control synthesis problems. Therefore, we will explore the structure of the corresponding value functions while also enabling the efficient computation of lower bounds. More precisely, we will enable targeted pruning of the tensor and thereby improve computational efficiency to mitigate the curse of dimensionality.

Chapter Overview. In the remainder of this chapter, we will introduce the results of [13] in Sec. 4.2 with its connection to improving the scalability of abstraction-based approaches. We will also present simulation-relation-based work [1, 50, 55, 56] in Sec. 4.3 that considers more uncertainty information. Specifically, this line of work could be the enabling layer for WP4. WP4 increases the system's knowledge awareness by injecting new data, context, and potentially broader uncertainty sets. Our abstraction and simulation-relation framework is built precisely for that regime: it turns the possibly evolving knowledge from WP4 into calibrated risk budgets and updated satisfaction-probability guarantees. In effect, WP4 expands what is known; our methods quantify what that means for risk, continuously and provably. We also present in Sec. 4.3 our extension work of [13] making use of simulation relation to apply tensor computation approach to continuous state systems. In Sec. 4.4, we present our work in [57] that fuses discretization-based and discretization-free method to improve scalability.

4.1.1 Problem Statement

Our goal is to synthesize a control strategy for a gMDP $\mathbf{M}$ to ensure satisfaction of a given DFA specification ϕ. Correct-by-design methods for infinite-state stochastic systems typically rely on finite-state abstractions, which scale poorly. Approximate simulation relations [58, 2] are appealing because the explicit error bounds allow for trading output error against transition probability deviation. Additionally, extensions of simulation relations exist that work for model uncertainty. This work first mitigates the associated scalability issues of finite MDPs by exploiting the fully decoupled gMDPs.

As a continuation of Sec. 2.1, we review the problem statement as follows:

Problem statement 1: *Given a specification ϕ and a probability $p \in [0, 1]$, synthesize a control strategy for a set of fully decoupled **finite** gMDPs $\mathbf{M}^{(i)}$ such that the controlled system* (2.3) *satisfies ϕ with a lower bound, i.e.,*

$$\mathbf{P}(\mathbf{M} \times \mathbf{C}_{\boldsymbol{\pi}} \models \phi) \geq p. \tag{4.1}$$

To enable the usage for systems with continuous states and model uncertainty, we follow this up with looking at the effect of approximate simulation relations on the provable satisfaction probabilities.

Problem statement 2: *Given a specification ϕ and a probability $p \in [0, 1]$, synthesize a control strategy for a set of fully decoupled but **uncertain** gMDPs $\mathbf{M}^{(i)}$ such that the controlled system* (2.3) *satisfies ϕ with a lower bound.*

4.2 Tensor-based efficient risk quantification for finite MDPs

4.2.1 Value Iteration for Temporal Logic Specifications

scLTL as a probabilistic reach-avoid problem. The satisfaction of a scLTL specification ϕ can be rewritten to **a reach-avoid problem** [59] by leveraging Deterministic Finite State Automata (DFA).

Definition 6 (DFA)**.** *A deterministic finite automaton is defined by the tuple $\mathcal{A} = (Q, q_0, \Sigma_{\mathcal{A}}, \tau_{\mathcal{A}}, q_f)$. Here, Q, q_0, and q_f denote the set of states, initial state, and accepting state, respectively. Furthermore, $\Sigma_{\mathcal{A}} = 2^{\mathrm{AP}}$ denotes the input alphabet and $\tau_{\mathcal{A}} : Q \times \Sigma_{\mathcal{A}} \to Q$ is a transition function.*

For a given word $\boldsymbol{l} = l_0, l_{1,2}, \ldots$, a run of a DFA defines a trajectory $q_0, q_1, \ldots, q_f$ that starts with q_0 and evolves according to $q_{t+1} = \tau_{\mathcal{A}}(q_t, l_t)$. The word $\boldsymbol{l}$ is accepted if the corresponding trajectory ends at q_f, that is $\exists t$ such that $q_t = q_f$. For any scLTL specification ϕ based on atomic propositions AP, there exists a DFA $\mathcal{A}_\phi$ with $\Sigma_{\mathcal{A}_\phi} = 2^{\mathrm{AP}}$ such that $\boldsymbol{l}$ satisfies the specification, that is $\boldsymbol{l} \models \phi$, if and only if $\boldsymbol{l}$ is accepted by $\mathcal{A}_\phi$. In the sequel, we will generally drop the subscript ϕ. Note that the specifications studied in this section satisfies the assumption as follows:

Assumption 3 (Independence requirement)**.** *Atomic propositions* AP *of the input alphabet $\Sigma_{\mathcal{A}}$ are composed of m disjoint sets, that is* $\mathrm{AP} := \cup_{i=1}^{m} \mathrm{AP}^{(i)}$, *with* $\mathrm{AP}^{(i)} \cap \mathrm{AP}^{(j)} = \emptyset$, *if* $i \neq j, \forall i, j \in \{1, 2, \ldots, m\}$. *Furthermore, there exist labeling maps $L^{(i)} : \mathbb{S}^{(i)} \to 2^{\mathrm{AP}^{(i)}}$ such that states $(s^{(1)}, \ldots, s^{(m)}) \in \prod_{i=1}^{m} \mathbb{S}^{(i)}$ are mapped to the sets of atomic propositions as*

$$L(s^{(1)}, s^{(2)}, \ldots, s^{(m)}) := \bigcup_{i=1}^{m} L^{(i)}(s^{(i)}) \in 2^{\mathrm{AP}}. \tag{4.2}$$

Consider a finite MDP $\mathbf{M} := (\mathbb{S}, \mathbb{A}, \mathbb{T}, s_0)$, a DFA $\mathcal{A} = (Q, q_0, \Sigma_{\mathcal{A}}, \tau_{\mathcal{A}}, q_f)$, and a labeling function $L : \mathbb{S} \to \Sigma_{\mathcal{A}}$. The product $\mathbf{M}_{\mathcal{A}}$ is a MDP

$$\mathbf{M}_{\mathcal{A}} := (\bar{\mathbb{S}}, \bar{\mathbb{A}}, \bar{\mathbb{T}}, \bar{s}_0) \tag{4.3}$$

where the initial state is given as $\bar{s}_0 := (s_0, \tau_{\mathcal{A}}(q_0, L(s_0)))$, and the state set $\bar{\mathbb{S}} = S \times Q$ is defined for elements $\bar{s} := (s, q)$, $\bar{\mathbb{A}} = \mathbb{A}$ is the input set, and $\bar{\mathbb{T}}(\cdot|\bar{s}, a)$ is the stochastic kernel that assigns for any state $\bar{s} = (s, q)$ and control a a probability to the next state $\bar{s}' = (s', q')$ given as $\bar{\mathbb{T}}(\bar{s}'|\bar{s}, a) = \mathbb{T}(s'|s, a)$ when $q' = \tau_{\mathcal{A}}(q, L(s'))$, and $= 0$ otherwise.

Define the accepting state set of the product model $\mathbf{M}_{\mathcal{A}_\phi}$ as $\bar{\mathbb{S}}_f := S \times \{q_f\}$. The controlled system $\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}$ satisfying an scLTL specification ϕ can be translated to a reachability problem with the target the accepting state set $\bar{\mathbb{S}}_f$. More precisely, the control synthesis problem can be defined as the design of a policy $\boldsymbol{\pi}$ such that the probability of reaching the accepting state of $\mathbf{M}_{\mathcal{A}_\phi}$ is optimized, that is,

$$\max_{\boldsymbol{\pi}} \mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \in \mathbb{N} \cup \infty : \bar{s}_t \in \bar{\mathbb{S}}_f). \tag{4.4}$$

This unbounded reach problem is the limit of the probabilistic bounded reachability

$$\begin{aligned} &\mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \in \mathbb{N} \cup \infty : \bar{s}_t \in \bar{\mathbb{S}}_f) \\ &\qquad := \lim_{N \to \infty} \mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \leq N : \bar{s}_t \in \bar{\mathbb{S}}_f). \end{aligned} \tag{4.5}$$

Dynamic Programming Mappings. We define value functions $\mathcal{V}^{\boldsymbol{\pi}}_{q,k} : \mathbb{S} \to [0,1]$ as the probability that the accepting state set $\bar{\mathbb{S}}_f := \mathbb{S} \times \{q_f\}$ is reached within k time steps starting from the state (s,q) in $\mathbf{M}_{\mathcal{A}_\phi}$. Given a Markov policy $\boldsymbol{\pi} := \{\pi[t] | t = 0, 1, \ldots, N-1\}$, value functions can iteratively be computed as

$$\mathcal{V}^{\boldsymbol{\pi}}_{q,k+1}(s) = \mathbf{1}_{\bar{\mathbb{S}}_f}(s,q) + \mathbf{1}_{\bar{\mathbb{S}} \setminus \bar{\mathbb{S}}_f}(s,q) \mathbb{E}^{s',q'}[\mathcal{V}^{\boldsymbol{\pi}}_{q',k}(s') | (s,q), a = \pi(s,q))] \tag{4.6}$$

with $\pi = \pi[N-1-k]$, $k \in \{0, \ldots, N-1\}$, and value functions initialized as

$$\begin{aligned} \mathcal{V}_{q_j,0}(s) &= 0, \text{ for } q_j \in Q \setminus \{q_f\}, \\ \mathcal{V}_{q_f,k}(s) &= 1, \text{ for } k \in \{0, 1, .., N-1\}. \end{aligned} \tag{4.7}$$

Proposition 1 (scLTL satisfaction)**.** *For a given policy $\boldsymbol{\pi} = \{\pi[k] | k = 0, 1, \ldots, N-1\}$ with $\pi[k] : \mathbb{S} \times Q \to \mathbb{A}$, the satisfaction probability within time horizon N for specification ϕ of system $\mathbf{M}$ is defined as*

$$\mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \leq N : \bar{s}_t \in \bar{\mathbb{S}}_f) := \mathcal{V}^{\boldsymbol{\pi}}_{\bar{q}_0,N}(s_0) \tag{4.8}$$

with $\bar{q}_0 = \tau_{\mathcal{A}}(q_0, L(s_0))$ and computed iteratively as in (4.6), and initialized as in (4.7).

Proposition 2 (Optimal scLTL satisfaction)**.** *For a bounded horizon N, the optimal satisfaction probability for specification ϕ of system $\mathbf{M}$ is computed iteratively as*

$$\mathcal{V}^{*}_{q,k+1}(s) = \mathbf{1}_{\bar{\mathbb{S}}_f}(s,q) + \mathbf{1}_{\bar{\mathbb{S}} \setminus \bar{\mathbb{S}}_f}(s,q) \mathbb{E}^{s',q'}[\mathcal{V}^{*}_{q',k}(s') | (s,q), a = \pi^*(s,q)] \tag{4.9}$$

based on the optimal policy $\pi^*(s,q)$, which is computed as

$$\pi^*(s,q) \in \arg\max_{\pi : \mathbb{S} \to \mathbb{A}} \mathbb{E}^{s',q'}[\mathcal{V}^{*}_{q',k}(s') | (s,q), a = \pi(s,q))].$$

Proposition 3. *For an infinite horizon, the optimal satisfaction probability for specification ϕ of system $\mathbf{M}$ is*

$$\max_{\boldsymbol{\pi}} \mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \in N \cup \infty : \bar{s}_t \in \bar{\mathbb{S}}_f) = \max\{\mathbf{1}_{q_f}(\bar{q}_0), \mathcal{V}^{*}_{\bar{q}_0,\infty}(s_0)\}, \tag{4.10}$$

where $\bar{q}_0 = \tau_{\mathcal{A}}(q_0, L(s_0))$, and $\mathcal{V}^{*}_{\bar{q}_0,\infty}(s_0)$ is the converged value function defined as

$$\mathcal{V}^{*}_{\bar{q}_0,\infty}(s_0) := \lim_{N \to \infty} \mathcal{V}^{*}_{\bar{q}_0,N}(s_0), \tag{4.11}$$

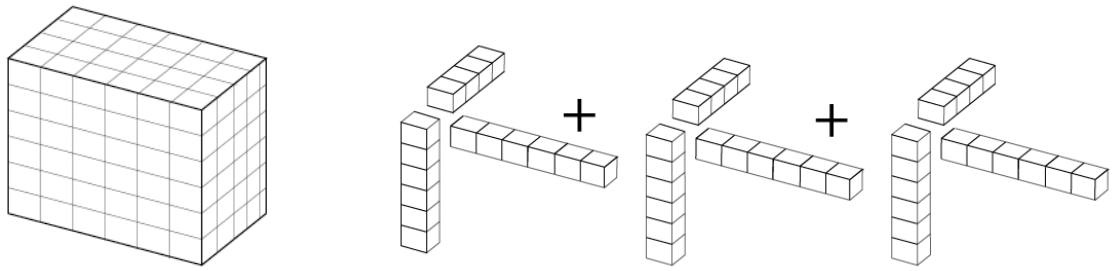
with initialization $\mathcal{V}^{*}_{\bar{q}_0,0}(s_0) = 0$, for $\bar{q}_0 \in Q \setminus \{q_f\}$, $\mathcal{V}^{*}_{\bar{q}_0,0}(s_0) = 1$, for $\bar{q}_0 \in \{q_f\}$. Additionally, the optimal policy is stationary: $\boldsymbol{\pi}^* = (\pi^*, \pi^*, \ldots)$.

4.2.2 Improved Scalability: Tensorized Dynamic Programming

Canonical Polyadic Decomposition. For $\mathbb{S} = \prod_{i=1}^{m} \mathbb{S}^{(i)}$, the value functions $\mathcal{V}^{\boldsymbol{\pi}}_{q,k} : \mathbb{S} \to [0,1]$, can be represented by an order-m tensor, which is a multi-dimensional array,

$$\mathbf{V} \in \mathbb{R}^{n_1 \times n_2 \times \ldots \times n_m} \tag{4.12}$$

with elements $\mathbf{V}_{i \ldots j} \in [0,1]$. For ease of reasoning we will use the function $\mathcal{V}^{\boldsymbol{\pi}}_{q,k}$ and its tensor representation $\mathbf{V}$ interchangeably. Based on Eq. (4.12) the memory required for the value function $\mathcal{V}^{\boldsymbol{\pi}}_{q,k}$ in (4.6) is $O(\prod_{i=1}^{m} n_i)$. However, some tensors can be represented more efficiently by their Canonical Polyadic Decomposition (CPD).



(a) Order-3 tensor with $6 \times 4 \times 6$ elements

(b) Illustration of Canonical Polyadic Decomposition of Order-3 tensor

Figure 4.1: Illustration of order-3 tensor with its Canonical Polyadic Decomposition

Definition 7 (CPD representation)**.** *Let* $\mathbf{V} \in \mathbb{R}^{n_1 \times n_2 \times \ldots \times n_m}$ *be an order-m tensor. If there exists a Canonical Polyadic Decomposition representation of* $\mathbf{V}$ *[60], i.e.,*

$$\mathbf{V} = \sum_{r=1}^{R} V_{\bullet r}^{(1)} \otimes V_{\bullet r}^{(2)} \otimes \ldots \otimes V_{\bullet r}^{(m)} \tag{4.13}$$

where $\otimes$ *denotes the outer product,* $V^{(k)}$ *is the factor matrix of size* $n_k \times R$ *for each mode* k*, and* $V_{\bullet r}^{(k)}$ *is the* r*-th column of* $V^{(k)}$*, then the CPD rank of* $\mathbf{V}$ *is* R *and is denoted as* $\mathrm{rank}(\mathbf{V})$*.*

As such for a CPD representation with rank R, the memory required is of the order $O(R \sum n_i)$. An order-3 tensor is visualized in Fig. 4.1, where the tensor rank is 3.

Boolean-formula-labelled Transitions of DFA. For each pair $(q, q^+) \in Q \times Q$, we define a set of Boolean subformulae $\alpha_1, \alpha_2, \ldots, \alpha_{n_\alpha}$ such that iff $l \models \bigvee_{i=1,\ldots,n_\alpha} \alpha_i$ then $\tau_{\mathcal{A}}(q, l) = q^+$. We require that the Boolean formulae are such that α_i is composed of a conjunction of (possibly negated) atomic propositions, that is $\alpha ::= p | \neg p | \alpha_1 \wedge \alpha_2$, and that for all $l \in \Sigma_{\mathcal{A}}$ there exists at most one $i \in \{1, \ldots, n_\alpha\}$ such that $l \models \alpha_i$. With a slight abuse of notation we extend the transitions to take these subformulae as arguments, that is $\tau_{\mathcal{A}}(q, \alpha_i) = q^+$. One can view a subformula being a subset of letters, that is $\alpha_i \subset 2^{\mathrm{AP}}$. We depict in Fig. 4.2 as an exemplary case. For any DFA $\mathcal{A}$ and a set of Boolean formulas, we represent the outgoing transitions of q using

$$\mathbf{N}_q := \{(\alpha_i, q') | q' = \tau_{\mathcal{A}}(q, \alpha_i)\}. \tag{4.14}$$

DFA-informed operators. We denote with $\boldsymbol{\pi}_q(s) = \boldsymbol{\pi}(s, q)$, and define a DFA-informed operator $\mathbf{T}_\alpha^{\pi_q}$ for the subformula[1] α and the policy $\pi_q : \mathbb{S} \to \mathbb{A}$:

$$\mathbf{T}_\alpha^{\pi_q}(\mathcal{V})(s) := \mathbb{E}^{s'}[\mathcal{L}_\alpha(s')\mathcal{V}(s') | s, a = \pi_q(s)], \tag{4.15}$$

with the next state s' evolving based on $\mathbb{T}$, and the indicator functions $\mathcal{L}_\alpha : \mathbb{S} \to \{0, 1\}$ defined for each subformula α as

$$\mathcal{L}_\alpha(s') = \begin{cases} 1, & \text{if } L(s') \models \alpha \\ 0, & \text{otherwise.} \end{cases} \tag{4.16}$$

[1]Note that in the remainder we will drop the indices of the subformulae α_i.

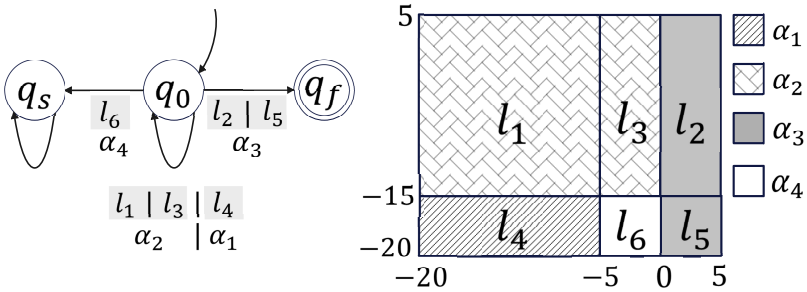


Figure 4.2: Left: DFA (q_s denotes q_{sink} for simplicity), right: state space labeled by l or α.

Note that the expected value in (4.6) is given over the next state s' and the next DFA state q'. However, since the DFA has deterministic dynamics in (4.15) the expected value only includes s'. We then rewrite the value iteration in (4.6) based on operator $\mathbf{T}_{\alpha}^{\pi_q}$ as

$$\mathcal{V}_{q,k+1}^{\boldsymbol{\pi}} = \sum_{(\alpha_i,q')\in\mathbf{N}_q} \mathbf{T}_{\alpha}^{\pi_q[N-1-k]}(\mathcal{V}_{q',k}^{\boldsymbol{\pi}}), \tag{4.17}$$

Lemma 2 (Equivalence of DFA-informed operator)**.** *Given a product model $\mathbf{M}_{\mathcal{A}}$ and a policy $\boldsymbol{\pi}$, the value functions $\mathcal{V}_{q,k+1}^{\boldsymbol{\pi}}$ and the corresponding probabilities of reaching the accepting state within time horizon N computed based on* (4.6)*(Proposition 1) are equal to those computed using DFA-informed operators as in* (4.17)*.*

Lemma 3. *Given a product model $\mathbf{M}_{\mathcal{A}}$, the optimal probabilities of reaching the accepting state within time horizon N computed based on* (4.9) *(Proposition 2) are equivalent to being computed using the proposed DFA-informed operators:*

$$\mathcal{V}_{q,k+1}^{*} = \max_{\pi_q[N-1-k]} \sum_{(\alpha,q')\in\mathbf{N}_q} \mathbf{T}_{\alpha}^{\pi_q[N-1-k]}(\mathcal{V}_{q',k}^{*})$$

The optimal policy for each mode q, i.e., $\boldsymbol{\pi}_q^* = (\pi_q^*[0], \pi_q^*[1], \pi_q^*[2], \ldots, \pi_q^*[N-1])$ can be computed as

$$\pi_q^*[N-1-k] \in \arg\max_{\pi_q[N-1-k]} \sum_{(\alpha,q')\in\mathbf{N}_q} \mathbf{T}_{\alpha}^{\pi_q[N-1-k]}(\mathcal{V}_{q',k}^{*}). \tag{4.18}$$

4.2.3 Computational Tensor Tree Expanded via $\mathbf{T}_{\alpha^{(i)}}^{\pi_q^{(i)}}$

We first review relevant concepts on graphs and trees [61]. A directed graph can be denoted by $\mathcal{G} = (\mathrm{N}, \mathcal{E})$, where $\mathrm{N} = \{1, \ldots, |\mathrm{N}|\}$ is the set of vertices and $\mathcal{E} \subset \mathrm{N} \times \mathrm{N}$ is the set of edges such that if $(i, j) \in \mathcal{E}$ then the graph contains a directed edge from vertex i to vertex j. Vertices, from which there is a directed edge to vertex j, are called *parents* of j. We call a vertex the *root* vertex if it has no parent, and from it all vertices in the graph can be reached via a sequence of directed edges. We call a directed graph a *rooted tree* if it consists of one root vertex and all the other vertices have one and only one parent. Additionally, we call vertices of a rooted tree with no outgoing edges *leaves*.

Definition 8 (Tree-based value function $\mathcal{G}$)**.** *A tree-based value function* $\mathcal{G} := \{\mathrm{N}, \mathcal{E}, v, \mathcal{L}_Q\}$ *has a set of vertices* N *and a set of (labelled) edges* $\mathcal{E} := \{(n, \alpha, n')\}$*, which define a* rooted tree*. Additionally, the vertex value mapping* $v : \mathrm{N} \to \mathbb{R}^{\prod |\mathbb{S}^{(i)}|}$ *maps a vertex* $n \in \mathrm{N}$ *to a tensor, i.e.,*

$$v(n) \in \mathbb{R}^{\prod |\mathbb{S}^{(i)}|},$$

and the DFA-state mapping $\mathcal{L}_Q : \mathrm{N} \to Q$ *maps a vertex* $n \in \mathrm{N}$ *to a DFA state* $q \in Q$*.*

Algorithm 2 Tree expansion $\mathcal{T}^\pi(\mathcal{G})$

```
procedure T^π(G)                                          ▷ Grow tree
    for n ∈ G.leaves do
        for {(q, α, q') ∈ τ_A | q' = L_Q(n)} do
            n̄ ← CreateNewVertex
            G.N ← N ∪ {n̄}
            G.E ← E ∪ {(n, α, n̄)}
            G.L_Q(n̄) ← q
        end for
    end for                                               ▷ Update tree values
    for (n, α, n') ∈ G.E do
        v(n') = T^π_α(v(n))
    end for
end procedure
```

Consider the initial tree-based value function $\mathcal{G}_0 = \{\mathrm{N}_0, \mathcal{E}_0, v_0, \mathcal{L}_{Q_0}\}$, with $\mathrm{N}_0 = \{1\}$, $\mathcal{E}_0 = \emptyset$, $v_0(1) = \mathbf{1}$, and $\mathcal{L}_Q(1) = q_f$. Based on $\mathcal{G}_0$, we can recover the initial value function $\mathcal{V}_{q,0}(s)$ as defined in (4.7). More precisely, $\mathcal{V}_{q_f,0}(s) = 1$ is represented in $\mathcal{G}_0$ by the tensor v_1 associated to vertex 1 in N_0. For a given Markov policy $\boldsymbol{\pi} = \{\pi[0], \ldots, \pi[N-1]\}$, the value iteration is equivalent to the

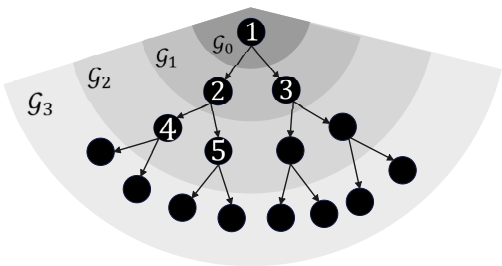


Figure 4.3: Tree expanded iteratively based on Alg. 2 as $\mathcal{G}_0$, $\mathcal{G}_1$, $\mathcal{G}_2$, and $\mathcal{G}_3$. Examples: $\mathcal{G}_0$ with $\mathrm{N}_0 = \{1\}$, $\mathcal{G}_1$ with $\mathrm{N}_1 = \{1, 2, 3\}$ and $\mathcal{E}_1 = \{(1, \alpha, 2), (1, \alpha, 3)\}$.

expansion of the tree based on operator $\mathcal{T}^\pi(\mathcal{G})$, which – as described in Alg. 2 – includes adding new leaves based on the transitions in the DFA and updating the value mapping v. More precisely, starting from $\mathcal{G}_0 = \{\mathrm{N}_0, \mathcal{E}_0, v_0, \mathcal{L}_{Q_0}\}$ the tree is iteratively grown or expanded

$$\mathcal{G}_{k+1} = \mathcal{T}^{\pi[N-1-k]}(\mathcal{G}_k). \tag{4.19}$$

The tree's growth is also visualized in Fig. 4.3.

Theorem 2 (Tensor rank of v_n)**.** *Given a tree $\mathcal{G}$ expanded based on Alg. 2 under Ass. 3 and with $\boldsymbol{\pi} \in \Pi_{dec}$, all vertices' values $v(n)$ with $n \in \mathrm{N}$ are of rank 1, that is,*

$$\begin{aligned} &\exists \mathrm{v}_n^{(1)} \in \mathbb{R}^{|\mathbb{S}^{(1)}|}, \mathrm{v}_n^{(2)} \in \mathbb{R}^{|\mathbb{S}^{(2)}|}, \dots \mathrm{v}_n^{(m)} \in \mathbb{R}^{|\mathbb{S}^{(m)}|} : \\ &v(n) = \mathrm{v}_n^{(1)} \otimes \mathrm{v}_n^{(2)} \otimes \dots \otimes \mathrm{v}_n^{(m)}. \end{aligned} \tag{4.20}$$

Theorem 3 (Tree mappings to $\mathcal{V}_{q,k}^{\boldsymbol{\pi}}$)**.** *For a horizon N, a Markov policy $\boldsymbol{\pi} = \{\pi[0], \dots, \pi[N-1]\} \in \Pi$ and a tree $\mathcal{G}_k = \{\mathrm{N}_k, \mathcal{E}_k, v_k, \mathcal{L}_{Q_k}\}$ computed with (4.19), we have that*

$$\mathcal{V}_{q,k}^{\boldsymbol{\pi}} = \sum_{n \in \mathcal{L}_{Q_k}^{-1}(q)} v_k(n), \tag{4.21}$$

where $\mathcal{L}_{Q_k}^{-1}(q) : Q \to \mathrm{N}$ gives the vertices set in $\mathcal{G}_k$ which is labeled q. (4.21) is $\mathbf{0}$ when $\{n | \mathcal{L}_Q(n) = q\} = \emptyset$.

4.2.4 Local Policy Optimization and Local Satisfaction Probabilities

For each discrete mode q, we want to update the policy locally. That is, we want to update the policy $\pi_q : \mathbb{S}^{(i)} \to \mathbb{A}$ for the i-th agent. More generally, we can also consider to design a stochastic policy $\pi_q : \mathbb{S}^{(i)} \times \mathbb{A} \to [0, 1]$.

For each mode q and each subsystem $i \in \{1, \dots, m\}$, we can develop efficient heuristic or approximate optimization of the policy. We use the following equation to find an optimized decoupled policy

$$\pi_q^{*(i)} \in \arg\max_{\pi_q^{(i)}} \sum_{e \in \mathcal{E}_q} \mathbf{T}_{\alpha^{(i)}}^{\pi_q^{(i)}}(v_n^{(i)}) c_e^{(i)} \tag{4.22}$$

where $\mathcal{E}_q := \{e = (n, \alpha, n') \in \mathcal{E} | \mathcal{L}_Q(n') = q\}$ and

$$c_e^{(i)} := \prod_{j}^{\{1,\dots,m\} \setminus \{i\}} \| \mathbf{T}_{\alpha^{(j)}}^{\pi_q^{(j)}}(v_n^{(j)}) \|_1.$$

$c_e^{(i)}$ is the heuristically approximated effects of other agents' state distributions. For now, it is assumed that the agents are uniformly distributed over the state space ($\|\cdot\|_1$). Alternative approaches could consider the use of sampling-based approximations and low-rank approximation of tensors as in [52, 53]. We remark that the above approach only delivers sub-optimal solutions, which is nevertheless often close to optimal behaviors.

Let μ be an assumed stationary distribution over the states that can be written as $\mu = \bigotimes_i^{\{1,\dots,m\}} \mu^{(i)}$. Then we can also express the local, per agent, satisfaction probability as

$$\mathcal{V}_q^{\boldsymbol{\pi},(i)} = \sum_{n \in \mathcal{L}_{Q_k}^{-1}(q)} \mathrm{v}_n^{(i)} \mathrm{c}_n^{(i)}, \tag{4.23}$$

with scalars $\mathrm{c}_n^{(i)}$ defined as

$$\mathrm{c}_n^{(i)} = \prod_{j}^{\{1,\dots,m\}} \mathrm{v}_j^{(1)} \mu^{(j)}. \tag{4.24}$$

In comparison to $\mathcal{V}_{q,k}^{\boldsymbol{\pi}}$, which cannot be evaluated for all states efficiently, $\mathcal{V}_q^{\boldsymbol{\pi},(i)}$ can be computed efficiently and can be represented based on the states of the i-th agent alone.

4.2.5 Scalability Benchmarking

The proposed tensor-tree-based method is equipped with formal guarantees, making it particularly suitable for safety-critical or high-assurance applications. The following results highlight its practical applicability and scalability. All simulations were run on a laptop computer with a 2.3 GHz 11th Gen Intel Core i7-11800H processor and 16.0 GB of RAM.

Two-dimensional Case. Consider an MDP that is abstracted via gridding discretization as in [2] from the stochastic difference equations

$$x_i^+ = 0.9x_i + 0.5u_i + w_i \text{ for } i = 1, 2 \tag{4.25}$$

with $w_i \sim \mathcal{N}(0, I)$ and with combined state $x = \begin{bmatrix} x_1 & x_2 \end{bmatrix}^T$.

The specification is $\psi := (\neg p_2 \wedge \neg p_3)\mathsf{U}p_1$. We visualize in Fig. 4.4 the satisfaction probability error computed as the difference between using exact value iteration and using rank-1 tree-based value iteration. In Fig. 4.4, it is shown that the maximum approximation error of satisfaction probabilities using rank-1 tree-based value iteration remains within the 10^{-2}. For each dimension gridded as 1000 cells, rank-1 tree-based value iteration reduces memory usage by 90% compared to exact value iteration, with only a 6% increase in computation time. We view this as a trade-off in small-scale problems, in the sense that we expect that in large-dimensional or multi-agent systems, rank-1 tree-based value iteration will be faster on average. We did not include comparisons in such settings, as exact value iteration becomes infeasible.

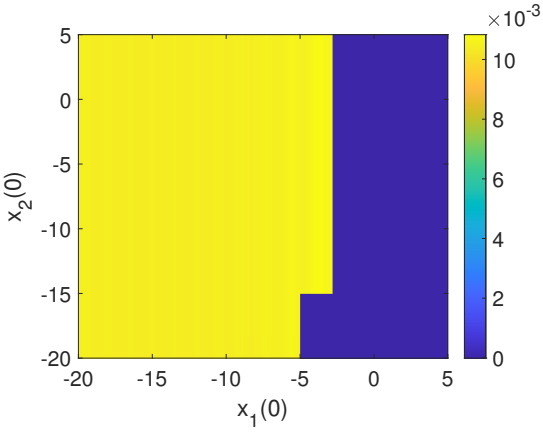


Figure 4.4: Satisfaction probability error of rank-1 tree-based value iteration. Error is computed with respect to using exact value iteration.

Four-dimensional Case. Consider an MDP that is composed from the stochastic difference equations for $i = 1, 2$

$$\mathbf{S}_i : \begin{aligned} \begin{bmatrix} x_1^+ \\ x_2^+ \end{bmatrix} &= \begin{bmatrix} 1 & t_s \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} u_1 + 0.5 \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \\ y_1 &= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \end{aligned}$$

with $t_s = 0.5$, $w_i \sim \mathcal{N}(0, 1)$, $\mathbb{X} = [-20, 5] \times [-5, 5] \times [-20, 5] \times [-5, 5]$, $\mathbb{U} = [-2, 2]^2$, $\mathbb{Y} = [-20, 5]^2$. The specification is $\psi := (\neg p_2 \wedge \neg p_3)\mathsf{U}p_1$, where the only difference is that p_1, p_2, and p_3 are atomic

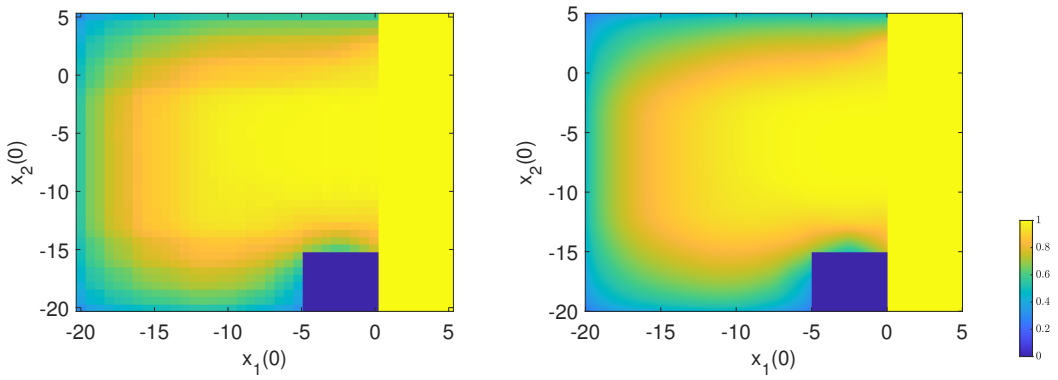


Figure 4.5: Effect of varying number of grid cells visualized as satisfaction probabilities on output space (left: 1.6×10^5 grid cells, right: 10^{12} grid cells).

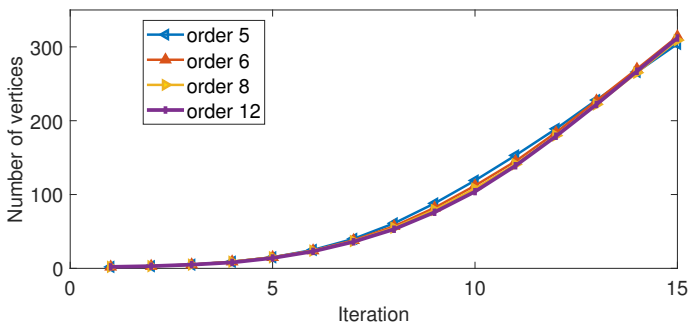


Figure 4.6: Expanding tree measured by the number of vertices at each iteration step. Each color corresponds to a choice of different order of magnitude in grid cell count.

propositions defined on output space $\mathbb{Y}$ instead of state space $\mathbb{X}$. We validate our tree pruning strategy by computing optimal satisfaction probabilities for a varying number of grid cells, that is $1.6 \times 10^5, 5.76 \times 10^6, 10^8$, and 10^{12}. In Fig. 4.5 we visualize such probabilities for state space being discretized as 1.6×10^5 grid cells and as 10^{12} grid cells. Running time and memory usage for all 4 cases are summarized in Table 4.1. With the number of grid cells increased by the order of 8 (from 10^5 to 10^{12}), the memory usage increases by 2.3×10^3 times (from 0.3482 MB to 799.8962 MB). In Fig. 4.6 we showcase that our pruning strategy keeps tree size (in terms of number of vertices) nearly unaffected by the exponential growth of MDP state space (in terms of number of grid cells).

High-dimensional Case. Consider an MDP that is composed from the stochastic difference equations

$$x_i^+ = x_i + u_i + w_i \tag{4.26}$$

with $w_i \sim \mathcal{N}(0, I)$, $\mathbb{X}^{(i)} : [-10, 10]$, $\mathbb{U}^{(i)} : [-2, 2]$. For scalability analysis purposes, we consider several case studies where the dimension d of the system state varies from 2 to 9, that is, from $i = 1, 2$ to $i = 1, \ldots, 9$. We consider two specifications: $\psi_1 := \bigwedge_{t=0}^{5} \bigcirc(\bigwedge_{i=1}^{d} p_1^{(i)})$ where $p_1^{(i)}$ is atomic proposition for which holds that $p_1^{(i)} = \text{true}$ iff $x_i \in [-5, 5]$, and $\psi_2 = \bigwedge_{i=1}^{d} p_1^{(i)} \mathsf{U} (\bigvee_{i=1}^{d} p_2^{(i)} \wedge \bigwedge_{i=1}^{d} p_1^{(i)})$, where $p_1^{(i)}, p_2^{(i)}$ are atomic propositions for which holds that $p_1^{(i)} = \text{true}$ iff $x_i \in [-5, 5]$, and $p_2^{(i)} =$

Table 4.1: Performance summary, including running time (in seconds) and memory usage (in MB) of varying system dimensions and grid counts for specification $\psi := (\neg p_2 \wedge \neg p_3) \mathsf{U} p_1$

d	Approach	Grid count	Time (s)	Memory (MB)
2	Exact VI	10^6	1.1866	753.3145
2	Full rank tree VI	10^6	19.2377	681.3138
2	Rank-1 tree VI	10^6	1.2626	96.2209
4	Rank-1 tree VI	1.6×10^5	19.2724	0.3482
4	Rank-1 tree VI	5.76×10^6	23.2937	1.9452
4	Rank-1 tree VI	10^8	28.6842	8.0142
4	Rank-1 tree VI	10^{12}	1085.6	799.8962

true iff $x_i \in [-2, 2]$. We consider grid cells up to 10^{27} for ψ_1, and grid cells up to 10^{18} for ψ_2. In Fig. 4.7, the computational time and memory requirements are plotted as functions of the number of agents. Each data point, indicated by the x marker, depicts either the computation time (shown in blue) or memory usage (shown in orange) for the corresponding case study. The results demonstrate that both memory usage and computation time exhibit sublinear scaling on a logarithmic scale for both specifications, with our approach capable of handling grid counts as large as 10^{27}, thereby underscoring its scalability in managing computational resources for large-sized problems. Further algorithmic enhancements could potentially improve this scalability, which we identify as a promising direction for future research.

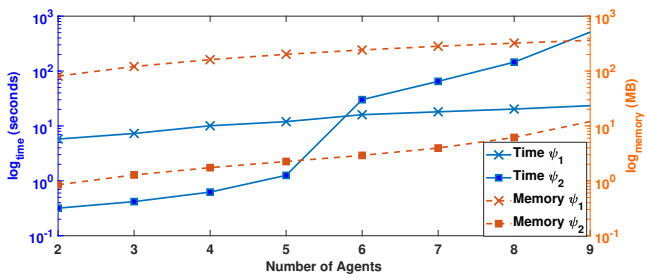


Figure 4.7: Scalability analysis (on running time in left y-axis as blue and memory usage in right y-axis as orange) for increasing number of agents. x represents time/memory for ψ_1, ■ for ψ_2.

4.3 Tensor-based Efficient Risk Quantification for Continuous State and Model Uncertainty

4.3.1 Simulation-Relation based Approaches for Uncertain Stochastic Systems

It is challenging to design control software embedded in safety-critical systems with guarantees of the satisfaction of temporal logic specifications. This is mainly due to the fact that most of these systems operate in an uncertain environment (i.e., their state evolution is subject to uncertainty) and that their state is comprised of both continuous and discrete variables. Unfortunately, most of the available results require prior knowledge of the exact stochastic model of the system, which

means any guarantee on the correctness of the closed-loop system only holds for that specific model. With the ever-increasing use of data-driven modeling and systems with learning-enabled components, we are able to construct accurate parameterized models with the associated confidence bounds. This includes confidence sets that capture the model uncertainty via either Bayesian inference or frequentist approaches.

Although existing results on similarity quantification can be extended to relate a model with a model set, these results would lead to controllers parameterized in a similar fashion as the model set. To design a single controller that works homogeneously for all models in the model set, a new type of simulation relation needs to be developed. In this line of work [1, 50, 55, 56], we provide an abstraction-based approach that is suitable for stochastic systems with model parametric uncertainty.

The considered set of models $\{\mathbf{M}(\theta) \text{ with } \theta \in \Theta\}$, is parameterized with θ:

$$\mathbf{M}(\theta) : \begin{cases} x_{k+1} & = f\left(x_k, u_k; \theta\right) + w_k \\ y_k & = h\left(x_k\right), \end{cases} \tag{4.27}$$

where the system state, input, and observation at the kth time-step are denoted by x_k, u_k, y_k. The functions f and h specify the parameterized state evolution of the system and the observation map. The additive noise is denoted by w_k, which is an independent, identically distributed noise sequence with distribution $w_k \sim p_w(\cdot)$. The uncertain parameter θ is assumed to belong to a known, bounded polytope $\Theta \in \mathbb{R}^p$ for some p. We assume that we are given an uncertainty set that contains the true parameters, and design a controller such that the controlled system satisfies a given temporal specification uniformly on this uncertainty set without knowing the true parameters. More specifically, we provide simulation relations for enabling correct-by-design control refinement, that are founded on coupling uncertainties of stochastic systems via sub-probability measures. Such relations are essential for constructing abstract models that are related to not only one model but to a set of parameterized models. We provide theoretical results for establishing this new class of relations and the associated closeness guarantees for both linear and nonlinear parametric systems with additive Gaussian uncertainty.

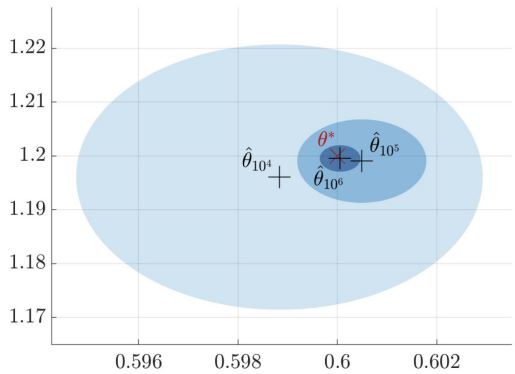


Figure 4.8: Contracting credible set Θ and parameter estimate $\hat{\theta}$ for data sizes $N = 10^4, 10^5, 10^6$ (light to dark blue) with an associated confidence bound of $(1 - \alpha) = 0.9$.

By developing robust (ϵ, δ) stochastic (sub)simulation relations that transfer temporal-logic guarantees from tractable abstractions to unknown systems, we produce explicit risk budgets and

uniform lower bounds on satisfaction over whole parameter sets, optionally with data-driven confidence from a credible set, as shown in Figure 4.8. This developed machinery is compositional (could scale to a network) and supports surrogate/contract-based environments, so as WP4 expands knowledge or exposes additional uncertainty, we simply update parameter credence or disturbance models and re-certify guarantees.

In the following sections, we show how the (ϵ, δ) accuracy information can be integrated in the tensor-based approach. We will focus on the integration of the δ error since the ϵ error can be integrated by leveraging more conservative labeling maps.

4.3.2 Tensor-Based Approach

Building upon previous results we have shown both in [13] and Sec. 4.2, we utilize simulation relations [2] to mitigate the computation scalability issue for correct-by-design control synthesis for continuous-state systems. We consider a set of abstract models $\mathbf{M}^{(i)}, i \in \{1, 2, \ldots, m\}$, each of which is $\delta^{(i)}$-stochastically simulated by $\mathbf{M}^{(i)}$.

Tensor Tree via Post-hoc Correction. We carry over the DFA-informed operator (4.15). Note that the computation is performed on each $\hat{\mathbf{M}}^{(i)}$, i.e., in (4.15) $\alpha := \alpha_i$, $\pi_q^g := \hat{\pi}_q^{(i)}$, $s^g := \hat{s}^{(i)}$, $s^{g'} := \hat{s}^{(i)'}$ and evolving based on $\hat{\mathbb{T}}^{(i)}$, and $a_g := \hat{a}^{(i)}$:

$$\mathbf{T}_{\alpha^{(i)}}^{\pi_q^{(i)}}(\mathrm{v}_n^{(i)})(s^{(i)}) := \mathbb{E}^{s'^{(i)}}[\mathcal{L}_{\alpha^{(i)}}(s'^{(i)})\mathrm{v}(s'^{(i)})|s^{(i)}, a^{(i)} = \pi_q^{(i)}(s^{(i)})]. \tag{4.28}$$

We show that the probabilistic deviation is relative to the *first hitting time*, of which the definition is as follows.

For executions of $\hat{\mathbf{M}}_{\mathcal{A}}$, the first *hitting time* of a set $A \subset \hat{\mathbb{S}} \times Q$ is a random variable conditioned on the initial state $(\hat{s}_0, \bar{q}_0)$ and given as

$$H_A(\hat{s}_0, \bar{q}_0) := \{\inf\{t \in \mathbb{N} \cup \{\infty\} : (\hat{x}_t, q_t) \in A\} \mid (\hat{s}_0, \bar{q}_0)\} \tag{4.29}$$

with $\bar{q}_0 = \tau_{\mathcal{A}}(q_0, L(\hat{s}_0))$. The support of first hitting time is $\mathbb{N} \cup \{\infty\}$. It can be infinity if the execution does not hit the set. It is 0 with probability 1 if $(\hat{s}_0, \bar{q}_0) \in A$, and is positive with probability 1 otherwise.

Proposition 4. *[First hitting time] The loss in probability by the sequential application of the robust operator* (4.15) *is related to the first hitting time of* $\hat{\mathbb{S}} \times \{q_f\}$ *in the product MDP* $\hat{M}_{\mathcal{A}}$ *as follows:*

$$\mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \leq l : \bar{s}_t \in \bar{\mathbb{S}}_f) \geq \mathcal{V}^l(\hat{s}, q) - \delta \sum_{h=1}^{l} \mathbb{P}(H_{\hat{\mathbb{S}} \times \{q_f\}}(\hat{s}, q) \geq h), \forall l \geq 1, \tag{4.30}$$

where $\delta := 1 - \prod_{i=1}^{m}(1 - \delta^{(i)})$.

Theorem 4 (Lower bounded satProb via post-hoc robustness correction)**.** *The loss in probability by the sequential application of the robust operator* (4.15) *is related to the tensor tree* $\mathcal{G}$ *expanded based on Alg. 2 as follows:*

$$\mathbf{P}_{\boldsymbol{\pi} \times \mathbf{M}_{\mathcal{A}_\phi}}(\exists t \leq l : \bar{s}_t \in \bar{\mathbb{S}}_f) \geq \mathcal{V}^l(\hat{s}, q) - l\delta + \delta \sum_{h=1}^{l} \sum_{i=0}^{h-1} \sum_{n \in \mathcal{D}_i^q} v(n) \tag{4.31}$$

where $v(n)$ is the value of the vertex that belongs to $\mathcal{D}_i^q$, and $\mathcal{D}_i^q$ the disjoint sets of vertices that share the same number of predecessors and the same q in $\mathcal{G}$, defined as

$$\mathcal{D}_i^q = \{n | n \text{ has exactly } i \text{ predecessors and } \mathcal{L}_Q(n) = q\}. \tag{4.32}$$

Robustness-aware Tensor Tree via On-the-fly Expansion. We introduce a $\delta^{(i)}$-robust operator as an extension of (4.15):

$$\mathbf{T}^{\pi_q^{(i)}}_{\delta^{(i)},\alpha^{(i)}}(\mathrm{v}_n^{(i)})(s^{(i)}) := \mathbf{L}\Big(\mathbb{E}^{s'^{(i)}}[\mathcal{L}_{\alpha^{(i)}}(s'^{(i)})\mathrm{v}_n^{(i)}(s'^{(i)})|s^{(i)}, a^{(i)} = \pi_q^{(i)}(s^{(i)})] - \delta^{(i)}\Big). \tag{4.33}$$

Note that we define the proposed operator explicitly for each $\mathbf{M}^{(i)}$ because $\delta^{(i)}$ is $\mathbf{M}^{(i)}$-specific; reusing the generic form (4.15) would obscure this dependence. Consequently, the tree is iteratively expanded as

$$\mathcal{G}_{k+1}^{\delta} = \mathcal{T}_{\delta}^{\pi[N-1-k]}(\mathcal{G}_k^{\delta}), \tag{4.34}$$

where $\mathcal{T}_{\delta}^{\pi[N-1-k]}$ is the operator described in Alg. 2, but with $\mathbf{T}_{\alpha}^{\pi}$ in line 11 replaced with $\mathbf{T}_{\delta^{(i)}}$(4.33).

Lemma 4 (Monotonicity of $\delta^{(i)}$-robust operator)**.** *Suppose $\hat{\mathbf{M}}^{(i)} \preceq_0^{\delta^{(i)}} \mathbf{M}^{(i)}$ with simulation relation $\mathcal{R}^{(i)}$ and a mapping $\pi_q^{(i)} : \hat{\mathbb{S}}^{(i)} \to \hat{\mathbb{A}}^{(i)}$ for all $i \in \{1, 2, \ldots, m\}$ is given. Let $\mathcal{V}_{||}(\hat{s}, s, q)$ and $\mathcal{V}(\hat{s}, q)$ have their tensor representations as*

$$\begin{aligned} \mathcal{V}_{||}(\hat{s}, s, q) &= \bigotimes_i \mathrm{v}_{||}(\hat{s}^{(i)}, s^{(i)}, q) \\ \mathcal{V}(\hat{s}, q) &= \bigotimes_i \mathrm{v}(\hat{s}^{(i)}, q), \end{aligned} \tag{4.35}$$

where $\mathrm{v}_{||}(\hat{s}^{(i)}, s^{(i)}, q) \geq \mathrm{v}(\hat{s}^{(i)}, q)$ for all $(\hat{s}^{(i)}, s^{(i)}) \in \mathcal{R}^{(i)}$, and for all i. Then for $(\hat{s}^{(i)}, s^{(i)}) \in \mathcal{R}^{(i)}$ and for $i \in \{1, 2, \ldots, m\}$,

$$\mathbf{T}^{\pi_q^{(i)}}_{\alpha^{(i)}}(\mathrm{v}_{||})(\hat{s}^{(i)}, s^{(i)}, q) \geq \mathbf{T}^{\pi_q^{(i)}}_{\delta^{(i)},\alpha^{(i)}}(\mathrm{v})(\hat{s}^{(i)}, q), \tag{4.36}$$

and

$$\mathbf{T}^{\pi_q}_{\alpha}(\mathcal{V}_{||})(\hat{s}, s, q) \geq \mathbf{T}^{\pi_q}_{\delta,\alpha}(\mathcal{V})(\hat{s}, q), \tag{4.37}$$

where $\mathbf{T}_{\alpha}^{\pi_q} := \bigotimes_i \mathbf{T}^{\pi_q^{(i)}}_{\alpha^{(i)}}$ is the operator defined with respect to stochastic transitions of the coupling similar model $\hat{\mathbf{M}}||_{\mathcal{R}}\mathbf{M} := \bigotimes_i \hat{\mathbf{M}}^{(i)}||_{\mathcal{R}^{(i)}}\mathbf{M}^{(i)}$, and $\mathbf{T}_{\delta,\alpha}^{\pi_q} := \bigotimes_i \mathbf{T}^{\pi_q^{(i)}}_{\delta^{(i)},\alpha^{(i)}}$ the robust operator with respect to the stochastic transitions of $\hat{\mathbf{M}} := \bigotimes_i \hat{\mathbf{M}}^{(i)}$, and $\mathbf{T}^{\pi_q^{(i)}}_{\alpha^{(i)}}$.

Theorem 5 (Lower bounded satProb via robustness-aware tensor tree)**.**

$$\mathbf{P}_{\boldsymbol{\pi}\times\mathbf{M}_{\mathcal{A}_\phi}}(\exists t \leq l : \bar{s}_t \in \bar{\mathbb{S}}_f) \geq \sum_{n \in \mathcal{L}_{Q_l}^{-1}(q)} v_l^{\delta}(n), \tag{4.38}$$

where $\mathcal{L}_{Q_l}^{-1}(q) : Q \to \mathrm{N}$ gives the vertices set in $\mathcal{G}_l^{\delta}$ which is labeled q. $\mathcal{G}_l^{\delta}$ is expanded based on Alg. 2 with $\mathbf{T}_{\alpha}^{\pi}$ in line 11 replaced with $\mathbf{T}^{\pi_q^{(1)}}_{\delta^{(1)},\alpha^{(1)}}, \mathbf{T}^{\pi_q^{(2)}}_{\delta^{(2)},\alpha^{(2)}}, \ldots, \mathbf{T}^{\pi_q^{(m)}}_{\delta^{(m)},\alpha^{(m)}}$ (4.33).

4.4 Complementary Work on Scalability: Heterogeneous Tooling

In this chapter, we have analyzed the scalability of tensor-based computation, and we have introduced simulation-relation-based framework that considers more uncertain systems. It is imperative that our risk assessment methodologies evolve in tandem. In this section, we enhanced the efficiency of our risk assessment methodology and hence improved the scalability of its application in mixed-autonomy environments. We tackled the problem of risk quantification for stochastic systems under temporal logic specifications by employing a sampling-based approach. Particularly, we utilized the SysCoRe toolbox, interfacing the sampling-based approach with the discretization-based approach through heterogeneous tooling to achieve our goal. By harnessing both the power of discretization-based and discretization-free approaches, we have created a flexible and adaptable framework capable of handling a wide range of risk scenarios. Especially, our integration of a sampling-based approach promotes scalability by allowing us to cherry-pick the most suitable approach for each risk assessment subtask. Our work creates a synergy that enhances the overall effectiveness and accuracy of risk assessment, which is expected to have industrial applications across various domains. Specifically, our work in [57] fuses discretization-based (DB) and discretization-free (DF) layers and ties them together with a single, specification-guided Bellman operator, so guarantees flow across models while each tool is used only where it is heuristically cheap and most accurate. Two engineering tricks were used to make it fast at scale: (i) compute the switching strategy on a coarse surrogate grid, then lift it to the fine DB model (huge cut in DP evaluations), and (ii) utilize a sampling-based strategy for unimportant regions.

In the large package delivery case, the heterogeneous *DB+DF* stack achieves a higher satisfaction bound than either single tool and slashes resources (5.05 s vs 14.4 s; 653 MB vs 2706 MB), illustrating scalability with concrete time/memory wins. Fig. 4.9 showcases the satisfaction probabilities computed using heterogeneous tooling.

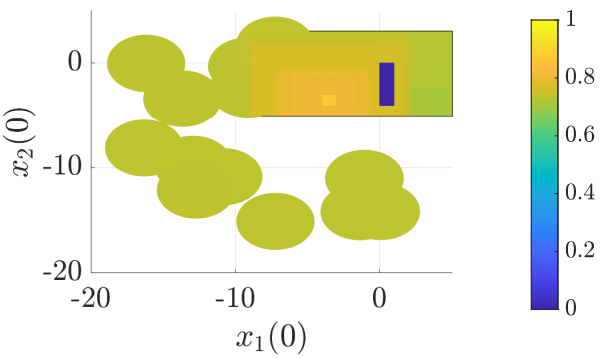


Figure 4.9: Robust satisfaction probability of package delivery case with enlarged spaces when using a multi-layered approach with heterogeneous layers.

4.5 Conclusions

In this chapter, we demonstrated how abstraction and simulation relations enable correct-by-design verification and synthesis for stochastic multi-agent systems, with policy refinement transferring

temporal-logic guarantees and quantified risk from finite models to their originals. We further exploited tensor structure to achieve scalable, compositional controller synthesis across large networks. To enhance practicality at scale, we introduced a specification-guided fusion of discretization-based and discretization-free layers. In the next chapter, we conclude the report.

Chapter 5

Conclusions

This report describes two approaches in which risks can be taken into account in multi-agent systems that need to satisfy linear temporal logic specifications. In Chapter 3, we give an abstraction-free approach for distributing temporal logic tasks. In Chapter 4, we give an abstraction-based approach that leverages low-rank tensors and that is also suitable for Markov decision models with uncertainty.

Bibliography

[1] O. Schön, B. van Huijgevoort, S. Haesaert, and S. Soudjani, "Correct-by-design control of parametric stochastic systems," in *2022 IEEE 61st Conference on Decision and Control (CDC)*. IEEE, 2022, pp. 5580–5587.

[2] S. Haesaert and S. Soudjani, "Robust dynamic programming for temporal logic control of stochastic systems," *IEEE Transactions on Automatic Control*, vol. 66, no. 6, pp. 2496–2511, 2020.

[3] M. H. W. Engelaar, S. Haesaert, and M. Lazar, "Stochastic model predictive control with dynamic chance constraints," in *International Conference on System Theory, Control and Computing*, 2023, pp. 356–361.

[4] A. Jones, M. Schwager, and C. Belta, "Distribution temporal logic: Combining correctness with quality of estimation," in *52nd IEEE Conference on Decision and Control*. IEEE, 2013, pp. 4719–4724.

[5] C.-I. Vasile, K. Leahy, E. Cristofalo, A. Jones, M. Schwager, and C. Belta, "Control in belief space with temporal logic specifications," in *2016 IEEE 55th Conference on Decision and Control (CDC)*. IEEE, 2016, pp. 7419–7424.

[6] V. I. Bogachev, *Measure theory*. Springer, 2007.

[7] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 1977, pp. 46–57.

[8] N. Piterman, A. Pnueli, and Y. Sa'ar, "Synthesis of reactive (1) designs," in *Verification, Model Checking, and Abstract Interpretation: 7th International Conference, VMCAI 2006, Charleston, SC, USA, January 8-10, 2006. Proceedings 7*. Springer, 2006, pp. 364–380.

[9] C. Belta, B. Yordanov, and E. A. Gol, *Formal methods for discrete-time dynamical systems*. Springer, 2017, vol. 89.

[10] O. Maler and D. Nicković, "Monitoring temporal properties of continuous signals," in *Proc. FORMATS / FTRTFT 2004 (Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems)*, ser. Lecture Notes in Computer Science, vol. 2791. Springer, 2004, pp. 152–166.

[11] L. Lindemann and D. V. Dimarogonas, "Decentralized robust control of coupled multi–agent systems under local signal temporal logic tasks," in *2018 Annual American Control Conference (ACC)*. IEEE, 2018, pp. 1567–1573.

[12] S. Qi, Z. Zhang, Z. Sun, and S. Haesaert, "Risk-aware autonomous driving with linear temporal logic specifications," *arXiv preprint arXiv:2409.09769*, 2024.

[13] R. Wang, Z. Sun, and S. Haesaert, "Unraveling tensor structures in correct-by-design controller synthesis," 2025. [Online]. Available: https://arxiv.org/abs/2503.24085

[14] Z. Zhang, Z. Sun, and S. Haesaert, "Intention-aware control based on belief-space specifications and stochastic expansion," *IEEE Transactions on Intelligent Vehicles*, 2024.

[15] V. Raman, A. Donzé, M. Maasoumy, R. M. Murray, A. Sangiovanni-Vincentelli, and S. A. Seshia, "Model predictive control with signal temporal logic specifications," in *Conference on Decision and Control*, 2014, pp. 81–87.

[16] D. Sun, J. Chen, S. Mitra, and C. Fan, "Multi-agent motion planning from signal temporal logic specifications," *Robotics and Automation Letters*, vol. 7, no. 2, pp. 3451–3458, 2022.

[17] S. Liu, A. Saoud, P. Jagtap, D. V. Dimarogonas, and M. Zamani, "Compositional synthesis of signal temporal logic tasks via assume-guarantee contracts," in *Conference on Decision and Control*, 2022, pp. 2184–2189.

[18] L. Lindemann and D. V. Dimarogonas, "Barrier function based collaborative control of multiple robots under signal temporal logic tasks," *Transactions on Control of Network Systems*, vol. 7, no. 4, pp. 1916–1928, 2020.

[19] T. Yang, Y. Zou, S. Li, and Y. Yang, "Distributed model predictive control for probabilistic signal temporal logic specifications," *Transactions on Automation Science and Engineering*, pp. 1–11, 2023.

[20] K. Braekers, K. Ramaekers, and I. Van Nieuwenhuyse, "The vehicle routing problem: State of the art classification and review," *Computers & industrial engineering*, vol. 99, pp. 300–313, 2016.

[21] M. Charitidou and D. V. Dimarogonas, "Signal temporal logic task decomposition via convex optimization," *Control Systems Letters*, vol. 6, pp. 1238–1243, 2021.

[22] K. Leahy, A. Jones, and C.-I. Vasile, "Fast decomposition of temporal logic specifications for heterogeneous teams," *Robotics and Automation Letters*, vol. 7, no. 2, pp. 2297–2304, 2022.

[23] Z. Zhang and S. Haesaert, "Modularized control synthesis for complex signal temporal logic specifications," in *Conference on Decision and Control*, 2023, pp. 7856–7861.

[24] H. Liu, P. Zhang, B. Hu, and P. Moore, "A novel approach to task assignment in a cooperative multi-agent design system," *Applied Intelligence*, vol. 43, pp. 162–175, 2015.

[25] D. P. Bertsekas, "Auction algorithms for network flow problems: A tutorial introduction," *Computational optimization and applications*, vol. 1, pp. 7–66, 1992.

[26] S. S. Farahani, R. Majumdar, V. S. Prabhu, and S. Soudjani, "Shrinking horizon model predictive control with signal temporal logic constraints under stochastic disturbances," *Transactions on Automatic Control*, vol. 64, no. 8, pp. 3324–3331, 2019.

[27] K. A. Mustafa, O. de Groot, X. Wang, J. Kober, and J. Alonso-Mora, "Probabilistic risk assessment for chance-constrained collision avoidance in uncertain dynamic environments," in *International Conference on Robotics and Automation*, 2023, pp. 3628–3634.

[28] M. H. W. Engelaar, Z. Zhang, M. Lazar, and S. Haesaert, "Risk-aware MPC for stochastic systems with runtime temporal logics," *arXiv preprint arXiv:2402.03165*, 2024.

[29] M. H. W. Engelaar, Z. Zhang, E. E. Vlahakis, D. V. Dimarogonas, M. Lazar, and S. Haesaert, "Risk-aware real-time task allocation for stochastic multi-agent systems under STL specifications," *arXiv preprint arXiv:2404.02111*, 2024.

[30] A. Donzé and O. Maler, "Robust satisfaction of temporal logic over real-valued signals," in *International Conference on Formal Modeling and Analysis of Timed Systems*, 2010, pp. 92–106.

[31] C. Burnett, T. J. Norman, and K. Sycara, "Trust decision-making in multi-agent systems," in *international joint conference on artificial intelligence*, 2011.

[32] B. C. van Huijgevoort, C. Verhoek, R. Tóth, and S. Haesaert, "Direct data-driven signal temporal logic control of linear systems," *arXiv preprint arXiv:2304.02297*, 2023.

[33] D. Bertsekas and S. E. Shreve, *Stochastic optimal control: the discrete-time case.* Athena Scientific, 1996, vol. 5.

[34] S. Dharmadhikari and K. Jogdeo, "Multivariate unimodality," *The Annals of Statistics*, pp. 607–613, 1976.

[35] S. Sadraddini and C. Belta, "Robust temporal logic model predictive control," in *Conference on Communication, Control, and Computing (Allerton)*, 2015, pp. 772–779.

[36] L. Hewing and M. N. Zeilinger, "Stochastic model predictive control for linear systems using probabilistic reachable sets," in *Conference on Decision and Control*, 2018, pp. 5182–5188.

[37] K. Leahy, M. Mann, and C.-I. Vasile, "Rewrite-based decomposition of signal temporal logic specifications," in *NASA Formal Methods Symposium*, 2023, pp. 224–240.

[38] G. E. Fainekos and G. J. Pappas, "Robustness of temporal logic specifications for continuous–time signals," *Theoretical Computer Science*, vol. 410, no. 42, pp. 4262–4291, 2009.

[39] G. K. Venayagamoorthy, "Dynamic, stochastic, computational, and scalable technologies for smart grids," *IEEE Computational Intelligence Magazine*, vol. 6, no. 3, pp. 22–35, 2011.

[40] K. D. Julian, M. J. Kochenderfer, and M. P. Owen, "Deep neural network compression for aircraft collision avoidance systems," *Journal of Guidance, Control, and Dynamics*, vol. 42, no. 3, pp. 598–608, 2019.

[41] D. Zhang, G. Feng, Y. Shi, and D. Srinivasan, "Physical safety and cyber security analysis of multi-agent systems: A survey of recent advances," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 2, pp. 319–333, 2021.

[42] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2016.

[43] M. H. Engelaar, M. P. Swaanen, M. Lazar, and S. Haesaert, "Stochastic mpc for finite gaussian mixture disturbances with guarantees," *arXiv preprint arXiv:2411.07887*, 2024.

[44] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, "Learning-based model predictive control: Toward safe learning in control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, no. 1, pp. 269–296, 2020.

[45] P. Tabuada, *Verification and control of hybrid systems: a symbolic approach.* Springer Science & Business Media, 2009.

[46] L. Lamport, "What good is temporal logic?" in *IFIP congress*, vol. 83, 1983, pp. 657–668.

[47] S. Liu, N. Noroozi, and M. Zamani, "Symbolic models for infinite networks of control systems: A compositional approach," *Nonlinear Analysis: Hybrid Systems*, vol. 43, p. 101097, 2021.

[48] K. Mallik, A.-K. Schmuck, S. Soudjani, and R. Majumdar, "Compositional synthesis of finite-state abstractions," *IEEE Transactions on Automatic Control*, vol. 64, no. 6, pp. 2629–2636, 2018.

[49] A. Lavaei, S. Soudjani, and M. Zamani, "Compositional abstraction-based synthesis of general mdps via approximate probabilistic relations," *Nonlinear Analysis: Hybrid Systems*, vol. 39, p. 100991, 2021.

[50] O. Schön, B. van Huijgevoort, S. Haesaert, and S. Soudjani, "Verifying the unknown: Correct-by-design control synthesis for networks of stochastic uncertain systems," in *2023 62nd IEEE Conference on Decision and Control (CDC).* IEEE, 2023, pp. 7035–7042.

[51] A. Gorodetsky, S. Karaman, and Y. Marzouk, "High-dimensional stochastic optimal control using continuous tensor decompositions," *The International Journal of Robotics Research*, vol. 37, no. 2-3, pp. 340–377, 2018.

[52] S. Rozada and A. G. Marques, "Tensor low-rank approximation of finite-horizon value functions," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).* IEEE, 2024, pp. 5975–5979.

[53] H. Y. Ong, "Value function approximation via low-rank models," *arXiv preprint arXiv:1509.00061*, 2015.

[54] J. I. Alora, A. Gorodetsky, S. Karaman, Y. Marzouk, and N. Lowry, "Automated synthesis of low-rank control systems from sc-ltl specifications using tensor-train decompositions," in *2016 IEEE 55th Conference on Decision and Control (CDC).* IEEE, 2016, pp. 1131–1138.

[55] O. Schön, B. van Huijgevoort, S. Haesaert, and S. Soudjani, "Bayesian formal synthesis of unknown systems via robust simulation relations," *IEEE Transactions on Automatic Control*, 2024.

[56] O. Schön, S. Haesaert, and S. Soudjani, "Formal control for uncertain systems via contract-based probabilistic surrogates (extended version)," *arXiv preprint arXiv:2506.16971*, 2025.

[57] B. C. van Huijgevoort, R. Wang, S. Soudjani, and S. Haesaert, "Specification-guided temporal logic control for stochastic systems: a multi-layered approach," *arXiv preprint arXiv:2407.03896*, 2024.

[58] S. Haesaert, S. E. Zadeh Soudjani, and A. Abate, "Verification of general markov decision processes by approximate similarity relations and policy refinement," *SIAM Journal on Control and Optimization*, vol. 55, no. 4, pp. 2333–2367, 2017.

[59] A. Abate, M. Prandini, J. Lygeros, and S. Sastry, "Probabilistic reachability and safety for controlled discrete time stochastic hybrid systems," *Automatica*, vol. 44, no. 11, pp. 2724–2734, 2008.

[60] F. L. Hitchcock, "The expression of a tensor or a polyadic as a sum of products," *Journal of Mathematics and Physics*, vol. 6, no. 1-4, pp. 164–189, 1927.

[61] R. B. Bapat, *Graphs and matrices*. Springer, 2010, vol. 27.